UNIVERSIDAD FRANCISCO DE VITORIA

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA INFORMÁTICA

PROYECTO FINAL DE GRADO

MODALIDAD INGENIERÍA

Sistema de detección de imágenes inapropiadas en el navegador: Limp.IA

Cosme Morillo Juliá

Convocatoria de julio 2025

CALIFICACIÓN DEL PROYECTO FINAL DE GRADO

CUALITATIVA:	
NUMÉRICA:	

Conforme Presidente:	Conforme Secretario:
Fdo.:	Fdo.:

Conforme Vocal:	Conforme Vocal:	Conforme Vocal:
Fdo.:	Fdo.:	Fdo.:

Lugar y fecha: Pozuelo de Alarcón, a ____ de _____ de 202__

Queremos mirar con ojos limpios, animados por la predicación del Maestro: bienaventurados los que tienen puro su corazón, porque ellos verán a Dios.

- *San Josemaría* [1]

A mi familia. A todos los que quieren tener una mirada limpia y amar a los demás con un corazón grande.

Agradecimientos

Quiero agradecer a mi familia por haberme apoyado siempre en todas las decisiones que he tomado. Por estar en los días buenos y en los malos, siempre dispuestos a sacar una sonrisa. A mis amigos, por estar ahí siempre y por sacar lo mejor de mí. A mis profesores de la universidad y a los del colegio, porque de ellos he aprendido mucho, no solo en el ámbito académico, sino también en el personal. Han confiado en mí y han tenido una gran influencia en la persona que soy hoy.

En especial quiero agradecer a Manu, Javi y Jero por todo lo que me han aportado y ayudado a crecer. También quiero agradecer a mi tutor de PFG, Juan, por haberme acompañado durante este año y haber arrimado el hombro en los momentos difíciles.

Resumen

Este documento describe el Proyecto Final de Grado que desarrolla Limp.IA. Una extensión de navegador que detecta y difumina imágenes de contenido sugerente en tiempo real, con el objetivo de proteger a los usuarios de una exposición no deseada. La memoria abarca desde el análisis ético y legal del uso de inteligencia artificial en este ámbito hasta la implementación técnica del sistema, basado en redes neuronales convolucionales entrenadas para esta tarea. Se ha diseñado una arquitectura propia optimizada para su integración en Firefox, alcanzando más de un 94 % de acierto en la clasificación. El proyecto también reflexiona sobre el valor social de la herramienta y la necesidad de un desarrollo tecnológico orientado al bien común. Limp.IA pretende ser una herramienta sencilla y eficaz que contribuya a un uso más seguro y responsable de Internet.

Palabras claves

Inteligencia artificial, extensión de navegador, ética digital, redes neuronales convolucionales, bienestar digital.

Abstract

This document describes the Final Degree Project that develops Limp.IA. A browser extension that detects and blurs suggestive content images in real time, with the aim of protecting users from unwanted exposure. The report covers from the ethical and legal analysis of the use of artificial intelligence in this field to the technical implementation of the system, based on convolutional neural networks trained for this task. A proprietary architecture has been designed and optimized for integration into Firefox, achieving more than 94% accuracy in classification. The project also reflects on the social value of the tool and the need for a technological development oriented to the common good. Limp.IA aims to be a simple and effective tool that contributes to a safer and more responsible use of the Internet.

Keywords

Artificial intelligence, browser extension, digital ethics, convolutional neural networks, digital well-being

Índice de Contenidos

1. Introducción 1

2. Investigación previa 3

2.1. Soluciones existentes 3

2.1.1. PorNo! *Porn Blocker* 3

2.1.2. Smart Internet Porn Filter from vRate 3

2.1.3. Google SafeSearch 3

2.1.4. Microsoft Azure Content Moderator 4

2.1.5. NSFW Content Detection for Images and Videos - Sightengine 4

2.1.6. NSFW JS 4

2.2. Introducción a las CNNs 4

2.2.1. Las Redes Neuronales dentro de la Inteligencia Artificial 4

2.2.2. Redes Neuronales Convolucionales 5

2.2.2.1. Diferencias con Redes Neuronales Tradicionales 5

2.2.3. Funcionamiento Interno de una CNN 5

2.2.3.1. Capas de una CNN 6

2.2.4. Aplicaciones de las CNNs 6

3. Objetivos 7

3.1. Objetivo general 7

3.2. Lista de objetivos específicos 7

3.3. Métodos de Validación 8

4. Plan de Desarrollo del Proyecto 9

4.1. Metodología 9

4.2. Tecnologías 9

4.2.1. Tecnologías web y desarrollo en Firefox 9

4.2.2. Modelo de IA 10

4.2.3. Desarrollo del modelo en Python 10

4.2.4. *Dataset* para el entrenamiento 10

4.3. Plan de Trabajo 12

4.4. Recursos. 15
4.4.1. Recursos humanos 15
4.4.2. Recursos materiales 15
4.4.3. Recursos de software 15
4.5. Costes 16
4.6. Condicionantes y Limitaciones 17

5. Desarrollo de la Solución Técnica 19
5.1. Investigación 19
5.2. Selección de Datos 20
5.3. Desarrollo 21
5.3.1. Desarrollo del modelo 21
5.3.1.1. Explicación de técnicas utilizadas 21
5.3.1.2. Evaluación comparativa y elección del modelo final 23
5.3.2. Desarrollo de la extensión 27
5.4. Integración del Modelo en la Extensión 29
5.5. Pruebas 30
5.6. Revisión de la documentación 30

6. Resultados 31
6.1. Verificación de objetivos 31
6.2. Verificación de requisitos 33

7. Implicaciones Éticas e Impacto Social 37
7.1. Introducción 37
7.1.1. Marco ético y legal 37
7.2. Desarrollo 38
7.2.1. Valor del proyecto 38
7.2.2. Alcance del proyecto 38
7.2.3. Responsabilidad e impacto 39
7.3. Conclusiones 39

8. Mi Recorrido en la UFV 41
8.1. El PFG como culminación de mi camino universitario 41
8.2. Vinculación con mi futuro profesional 42

9. Conclusiones 43

10. Otros Méritos del Proyecto 45

11. Bibliografía 47

Anexo A: Manual de instalación 52

Anexo B: Requisitos 56

Anexo C: Desarrollo 59

Anexo D: Pruebas 62

Índice de Tablas

Tabla 1: Paquete de trabajo 1. Investigación – Elaboración propia *12*

Tabla 2: Paquete de trabajo 2. Seleción de Datos – Elaboración propia *12*

Tabla 3: Paquete de trabajo 3. Desarrollo – Elaboración propia *12*

Tabla 4: Paquete de trabajo 4. Integración – Elaboración propia *13*

Tabla 5: Paquete de trabajo 5. Pruebas – Elaboración propia *13*

Tabla 6: Paquete de trabajo 6. Documentación – Elaboración propia *13*

Tabla 7: Costes globales – Elaboración propia *16*

Tabla 8: Costes de Personal – Elaboración propia. *16*

Tabla 9: Parámetros data augmentation – Elaboración propia. *22*

Tabla 10: Parámetros reducción de LR – Elaboración propia. *22*

Tabla 11: Parámetros de parada temprana – Elaboración propia. *23*

Tabla 12: Hiperparámetros del modelo – Elaboración propia. *24*

Tabla 13: Valores de las métricas – Elaboración propia. *25*

Tabla 14: Valores de métricas K-Fold – Elaboración propia. *27*

Tabla 15: Grado de cumplimiento de los RF – Elaboración Propia *34*

Tabla 16: Grado de cumplimiento de los RNF – Elaboración Propia *35*

Tabla 17: : Requisitos Funcionales – Elaboración Propia *56*

Tabla 18: Requisitos No Funcionales– Elaboración Propia *58*

Índice de Figuras

Figura 1: Arquitectura de CNNs – Fuente: [16].....5
Figura 2: Plan de trabajo general – Elaboración propia.....13
Figura 3: Plan de trabajo detallado – Elaboración propia.....14
Figura 4: Arquitectura del modelo – Elaboración propia......24
Figura 5: Evolución de la precisión – Elaboración propia......26
Figura 6: Evolución de la pérdida – Elaboración propia......26
Figura 7: Versión beta de la extensión – Elaboración propia......28
Figura 8: Popup de la extensión – Elaboración propia......28
Figura 9: Limp.IA integrado en el navegador – Fuente: [29].....32
Figura 10: Limp.IA difuminando en web – Fuente: [30].....33
Figura 11: Abrir Firefox – Elaboración propia.....52
Figura 12: Buscar en la barra de direcciones – Elaboración propia.....53
Figura 13: Menu lateral – Elaboración propia.....53
Figura 14: Cargar extensión – Elaboración propia.....53
Figura 15: Seleccionar manifest.json – Elaboración propia.....54
Figura 16: Extensión cargada correctamente – Elaboración propia.....54
Figura 17: Icono de extensiones Firefox – Elaboración propia.....55
Figura 18: Fijar icono de extensión – Elaboración propia.....55
Figura 19: Popup activo y fijado en la barra de herramientas – Elaboración propia.....55
Figura 20: Muestra representativa clase sexy – Elaboración propia......59
Figura 21: Muestra representativa clase neutral – Elaboración propia.....60
Figura 22: Icono Limp.IA – Elaboración propia......60
Figura 23: Icono simplificado Limp.IA – Elaboración propia......60
Figura 24: Matríz de confusión – Elaboración propia......61
Figura 25: Evolución del LR – Elaboración propia......61
Figura 26: Limp.IA integrado en el navegador (mostrando score) – Fuente: [29].....62
Figura 27: Limp.IA en la página web oficial de Mango – Fuente: [34].....63
Figura 28: Ejemplo de imagen difuminada (mostrando score) – Fuente: [35].....63
Figura 29: Imagen no difuminada – Fuente: [36].....64
Figura 30: Imagen no difuminada mujer – Fuente: [37].....64
Figura 31: Ejemplo imagen protegida (CORS) – Fuente: [38].....65
Figura 32: Ejemplo imagen no clasificada – Fuente: [39].....65

Lista de Acrónimos

Acrónimo	**Significado**
API	*Application Programming Interface. Interfaz de Programación de Aplicaciones*
BIN	*Binary file*. Archivo binario
CNN	*Convolutional Neural Network.* Red Neuronal Convolucional
CORS	*Cross-Origin Resource Sharing.* Mecanismo que permite o restringe el acceso a recursos web desde dominios distintos al original.
CSS	*Cascading Style Sheets.* Hojas de Estilo en Cascada
DOM	*Document Object Model.* Modelo jerárquico que representa los elementos de una página web
Git	Sistema de control de versiones distribuido
HTML	*HyperText Markup Language*. Lenguaje de Marcado de Hipertexto
IA	Inteligencia Artificial
JS	JavaScript
JSON	*JavaScript Object Notation*. Notación de Objetos de JavaScript
K-Fold	Validación cruzada en K particiones
LR	*Learning Rate*. Tasa de aprendizaje
NSFW	*Not Safe For Work.* No seguro para el trabajo (contenido sensible/inapropiado)
OCR	*Optical Character Recognition*. Reconocimiento Óptico de Caracteres
PFG	Proyecto Final de Grado
RF	Requisito Funcional
RNF	Requisito No Funcional
ReLU	*Rectified Linear Unit.* Unidad Lineal Rectificada (función de activación)
URL	*Uniform Resource Locator.* Localizador Uniforme de Recursos
V2	Versión 2
V3	Versión 3
VS	Visual Studio

1. INTRODUCCIÓN

El desarrollo de herramientas tecnológicas orientadas al bien común es uno de los grandes retos de la ingeniería informática contemporánea. En un contexto digital marcado por el exceso de exposición a contenido visual y la ausencia de mecanismos efectivos de control, este Proyecto Final de Grado aborda una problemática especialmente sensible: la detección automática de imágenes inapropiadas en entornos web, con el fin de proteger tanto a los profesionales expuestos como a los usuarios más vulnerables.

La inspiración para este trabajo nace de dos preocupaciones sociales de gran relevancia. Por un lado, el impacto psicológico que sufren quienes se dedican a la moderación manual de contenido sensible en plataformas digitales. Casos documentados [2], como el de un trabajador de CCC Barcelona Digital Services (empresa subcontratada por Meta), que desarrolló graves trastornos mentales tras revisar contenido explícito durante dos años, evidencian la necesidad urgente de soluciones automatizadas que reduzcan la exposición directa de personas a este tipo de materiales.

Por otro lado, preocupa profundamente la creciente exposición y adicción al contenido pornográfico en edades tempranas. Según la campaña “Vamos a hablar de pornografía” [3], el 90 % de los adolescentes ha estado expuesto a este tipo de contenidos, en algunos casos desde los ocho años. A esta realidad se suman los hallazgos de investigaciones recientes que alertan sobre el profundo impacto de la exposición involuntaria a material sexual explícito en el desarrollo psicosexual de los menores.

Un estudio realizado en Castellón (España) [4] revela que el 85,9 % de los adolescentes entre 13 y 17 años ha estado expuesto de manera no intencionada a este tipo de materiales, con una edad media de primera exposición de solo 12,1 años. Este contacto prematuro se asocia a consecuencias psicológicas significativas, como síntomas de evitación emocional, aumento de activación fisiológica, reexperimentación del suceso y alteraciones en el desarrollo sexual, especialmente cuando la exposición ocurre a edades más tempranas o ante determinados tipos de contenido.

Estos datos refuerzan la urgencia de desarrollar sistemas tecnológicos eficaces que permitan proteger a los menores frente a contenidos potencialmente traumáticos, contribuyendo a un entorno digital más seguro.

Este proyecto propone una solución tecnológica basada en redes neuronales convolucionales (CNNs), entrenadas para clasificar imágenes como apropiadas o sensibles, e integradas en una extensión funcional para el navegador Firefox. El sistema, denominado Limp.IA, actúa en tiempo real difuminando imágenes cuya probabilidad de ser inapropiadas supera un umbral

configurable, todo ello sin necesidad de conexión externa ni envío de datos, garantizando la privacidad del usuario.

La memoria se compone de once capítulos, organizados de forma progresiva para guiar al lector desde el contexto inicial hasta la solución implementada y sus resultados. Tras esta introducción, el capítulo 2 presenta el marco teórico y los antecedentes, analizando tanto soluciones existentes como los fundamentos técnicos de las CNNs. En el capítulo 3 se definen los objetivos generales y específicos del proyecto, junto con los métodos de validación utilizados para evaluar su cumplimiento. El capítulo 4 describe la planificación temporal, la metodología adoptada y las tecnologías empleadas en el desarrollo. A continuación, el capítulo 5 expone en detalle el proceso de implementación técnica, desde el entrenamiento del modelo de IA hasta su integración en la extensión para navegador.

El capítulo 6 recoge los resultados obtenidos y evalúa el rendimiento del sistema a partir de métricas y pruebas reales. El capítulo 7 aborda las implicaciones éticas y el impacto social del proyecto, destacando su relevancia en contextos de protección digital. En el capítulo 8, el autor reflexiona sobre su recorrido universitario y el papel que este proyecto ha tenido en su formación personal y profesional. El capítulo 9 presenta las conclusiones principales, y el capítulo 10 recoge otros méritos técnicos y humanos alcanzados durante el desarrollo. Finalmente, el capítulo 11 incluye la bibliografía consultada, y se añaden varios anexos con material complementario que documenta gráficamente el sistema y sus pruebas.

Este trabajo busca ser, por tanto, no solo un ejercicio técnico de aplicación de conocimientos adquiridos, sino también una aportación significativa y concreta a un problema real que afecta a millones de personas. A través del uso responsable de la inteligencia artificial, se plantea una herramienta práctica, ética y eficaz que contribuye a hacer de internet un espacio más seguro.

2. INVESTIGACIÓN PREVIA

Esta investigación se centrará en la exploración de distintas soluciones existentes en la moderación de contenido, analizando sus enfoques y limitaciones. Además, se estudiará la arquitectura de las CNNs y su aplicación en la clasificación de imágenes.

2.1. SOLUCIONES EXISTENTES

2.1.1. PorNo! *Porn Blocker*

PorNo! Porn Blocker [5] es una extensión para Google Chrome diseñada para bloquear el acceso a sitios web con contenido para adultos. Su funcionamiento se basa en el uso de listas negras de dominios previamente identificados. No utiliza inteligencia artificial ni análisis en tiempo real del contenido de las páginas, por lo que su efectividad depende directamente de la actualización de su base de datos de sitios prohibidos.

2.1.2. Smart Internet Porn Filter from vRate

El filtro inteligente de vRate [6] emplea un sistema de detección automática para impedir el acceso a sitios que contengan contenido pornográfico. Esta extensión analiza las páginas en tiempo real y bloquea aquellas que sean identificadas como inapropiadas, incluso si no están en una lista negra predefinida. Es probable que utilice reglas heurísticas, palabras clave y análisis semántico para determinar si una página debe ser restringida.

2.1.3. Google SafeSearch

Google SafeSearch [7] es una herramienta de filtrado de contenido que permite a los usuarios excluir de los resultados de búsqueda imágenes, videos y páginas web que contengan material explícito. Utiliza una combinación de listas negras, algoritmos de *machine learning* y técnicas de clasificación para detectar contenido inapropiado. SafeSearch analiza los metadatos de las páginas web, así como las imágenes y descripciones, para identificar material sensible. Aunque es una herramienta efectiva, su principal limitación es que solo actúa dentro del ecosistema de Google.

2.1.4. Microsoft Azure Content Moderator

Microsoft Azure Content Moderator [8]es una solución avanzada de moderación de contenido basada en inteligencia artificial que permite analizar imágenes, videos y texto para detectar material explícito o sensible. Su tecnología se basa en CNNs para la clasificación de imágenes y videos, mientras que para la detección de texto utiliza OCR y procesamiento de lenguaje natural. Este sistema permite no solo identificar imágenes de desnudos o violencia, sino también analizar textos en imágenes y extraer palabras ofensivas o sugerentes. [8]

2.1.5. NSFW Content Detection for Images and Videos - Sightengine

Sightengine [9]ofrece una API de detección de contenido NSFW que permite analizar imágenes y videos para identificar material explícito. Este sistema está basado en inteligencia artificial y utiliza redes neuronales profundas para categorizar contenido en diferentes niveles de sensibilidad, como desnudez parcial o total, contenido erótico, violencia y presencia de drogas. Sightengine se usa principalmente en plataformas de redes sociales, aplicaciones de mensajería y sitios con contenido generado por usuarios.

2.1.6. NSFW JS

NSFW JS [10]es una biblioteca de JavaScript que permite detectar imágenes NSFW directamente en el navegador sin necesidad de enviar datos a un servidor externo. Está basada en TensorFlow.js y utiliza un modelo de aprendizaje profundo para clasificar imágenes en diferentes categorías, como desnudos parciales, explícitos y contenido seguro.

2.2. INTRODUCCIÓN A LAS CNNS

Las CNN forman parte del campo de la IA , específicamente dentro del aprendizaje profundo; que es una rama del aprendizaje automático. Para entender la importancia de las CNNs, es fundamental situarlas dentro del contexto más amplio de la IA.

2.2.1. Las Redes Neuronales dentro de la Inteligencia Artificial

La inteligencia artificial es un campo de la informática que busca desarrollar sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el reconocimiento de patrones, la toma de decisiones y el procesamiento del lenguaje natural. Dentro de la IA, el aprendizaje automático permite a los sistemas mejorar su desempeño sin ser explícitamente programados, basándose en datos y patrones. [11]

Las redes neuronales artificiales son un subconjunto del aprendizaje automático inspiradas en la estructura del cerebro humano. Estas redes están compuestas por neuronas artificiales organizadas en capas que procesan y transforman la información mediante funciones matemáticas. [12]

2.2.2. Redes Neuronales Convolucionales

Las CNNs son una variante de las redes neuronales tradicionales diseñadas para procesar datos con una estructura en forma de cuadrícula, como las imágenes. Se distinguen por su capacidad para extraer automáticamente características espaciales relevantes mediante la aplicación de filtros. [13]

2.2.2.1. Diferencias con Redes Neuronales Tradicionales

Las redes neuronales tradicionales conectan cada neurona con todas las neuronas de la capa anterior y posterior. Esto genera una gran cantidad de parámetros, lo que hace difícil entrenar modelos cuando se trabaja con imágenes de alta resolución. [12]

Las CNNs, en cambio, introducen capas convolucionales, que aplican filtros a pequeñas regiones de la imagen en lugar de conectar cada píxel con todas las neuronas [14]. Esto reduce drásticamente la cantidad de parámetros y permite que el modelo aprenda características espaciales jerárquicas, desde bordes y texturas simples hasta estructuras complejas.

2.2.3. Funcionamiento Interno de una CNN

Las CNNs están compuestas por varias capas especializadas que extraen características de una imagen de manera progresiva. En las primeras capas, detectan patrones de bajo nivel como bordes y texturas. Posteriormente, las capas intermedias identifican características más abstractas, como partes de objetos. Finalmente, en las capas más profundas, la red es capaz de interpretar estos elementos y diferenciarlos dentro de categorías complejas, como rostros completos o distintos tipos de objeto [15]. A continuación, se presenta la arquitectura de la red, seguida de una explicación detallada de cada una de sus capas.

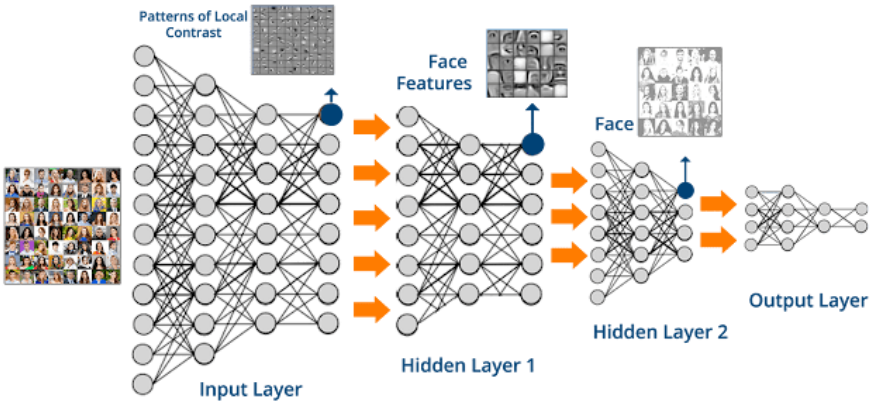

Figura 1: Arquitectura de CNNs – Fuente: [16]

2.2.3.1. Capas de una CNN

1. Capa de Convolución

Los filtros aplicados a la imagen de entrada permiten extraer características locales, funcionando como detectores de bordes, texturas y formas, lo que da lugar a un mapa de características en el que cada canal representa una transformación distinta de la imagen original. [13]

2. Capa de Activación

Introduce una transformación no lineal en los valores de las neuronas, permitiendo al modelo capturar relaciones complejas en los datos. Su función principal es mejorar la capacidad de aprendizaje al incorporar variaciones en la intensidad de las características detectadas.

3. Capa de *Pooling*

La capa de *pooling* se encarga de reducir la resolución de los mapas de características, lo que permite disminuir la cantidad de datos a procesar sin perder información relevante. Generalmente, se utiliza el *max pooling*, una técnica que selecciona el valor máximo dentro de una ventana determinada, ayudando a resaltar las características más importantes y a hacer la red más eficiente. [13]

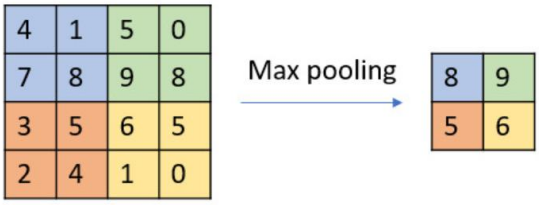

Figura 2 Funcionamiento de Max pooling – Fuente [16]

4. Capas Densas (Fully Connected Layers)

Reciben las características extraídas y las transforman en una representación que permite clasificar la imagen. De tal forma que actúan como un clasificador basado en las características aprendidas. [17]

5. Capa de Salida

Esta capa genera probabilidades asociadas a cada categoría en un problema de clasificación. [14]

2.2.4. Aplicaciones de las CNNs

Las CNNs destacan por su alto rendimiento en la detección y clasificación de imágenes, permitiendo identificar patrones visuales con gran precisión. Su capacidad para extraer características jerárquicas las hace ideales para el reconocimiento facial, la detección de objetos y la segmentación de imágenes. Son ampliamente utilizadas en seguridad, filtrado de contenido, automatización industrial y diagnóstico médico, donde su rapidez y precisión optimizan los procesos de análisis visual.

3. OBJETIVOS

3.1. OBJETIVO GENERAL

Desarrollar un sistema basado en redes neuronales que permita detectar y clasificar imágenes con contenido sensible o inapropiado en entornos web, con el fin de mejorar la seguridad y control de contenido en la navegación de los usuarios. Este sistema será integrado en una extensión de Firefox para su funcionamiento en tiempo real, garantizando precisión, eficiencia y cumplimiento de normativas éticas y de privacidad. Los requisitos funcionales y no funcionales que guían el desarrollo del sistema se detallan en el Anexo B: Requisitos

3.2. LISTA DE OBJETIVOS ESPECÍFICOS

1. **Seleccionar un *dataset* de calidad** que garantice un correcto entrenamiento y evaluación del modelo, asegurando diversidad y representatividad en las distintas categorías de clasificación.
2. **Diseñar y desarrollar la arquitectura de la red neuronal**, eligiendo el modelo más adecuado para la clasificación de imágenes, basado en un análisis comparativo de diferentes enfoques de aprendizaje profundo.
3. **Entrenar y optimizar la red neuronal**, aplicando técnicas de mejora de rendimiento como ajuste de hiperparámetros, aumento de datos o regularización para minimizar el sobreajuste.
4. **Evaluar el desempeño del modelo** mediante métricas como *precision*, *recall* o *F1-score*, utilizando un conjunto de pruebas independiente y aplicando técnicas de validación cruzada para garantizar su capacidad de generalización.
5. **Desarrollar la integración del modelo en una extensión**, asegurando su compatibilidad con el navegador y su correcto funcionamiento en la detección y filtrado de imágenes en tiempo real.
6. **Realizar pruebas de validación en entornos reales**, analizando la respuesta del sistema en distintos escenarios de navegación web y ajustando su funcionamiento para una experiencia óptima del usuario.

3.3. MÉTODOS DE VALIDACIÓN

1. **Selección del *dataset***

Se validará la calidad del *dataset* asegurando que contiene imágenes suficientes en cada categoría definida. Para ello, se realizará un análisis estadístico de la distribución de las imágenes con el fin de comprobar su representatividad y equilibrio en las clases. Además, se verificará la adecuación del *dataset* mediante una inspección manual de una muestra aleatoria de imágenes para garantizar que las etiquetas sean correctas.

2. **Diseño y desarrollo de la arquitectura de la red neuronal**

Se compararán diferentes arquitecturas de CNNs a través de un estudio de los distintos tipos de arquitecturas existentes. Asimismo, se analizarán las métricas de rendimiento en distintos modelos base para justificar la elección del modelo final.

3. **Entrenamiento y optimización del modelo**

Se medirán las mejoras en las métricas de rendimiento antes y después de aplicar técnicas de optimización como el ajuste de hiperparámetros. También se analizarán las curvas de aprendizaje y pérdida con el objetivo de garantizar que el modelo converge correctamente y no sufre de sobreajuste.

4. **Evaluación del desempeño del modelo**

Se utilizará un conjunto de prueba independiente para calcular las métricas de rendimiento. Adicionalmente, se aplicará validación cruzada con el propósito de evaluar la capacidad de generalización del modelo en diferentes subconjuntos de datos.

5. **Desarrollo e integración en una extensión**

Se validará la correcta implementación del modelo dentro del navegador. Para ello, se comprobará su funcionamiento e integración en tiempo real, asegurando su operatividad en distintos entornos de navegación.

6. **Pruebas de validación en entornos reales**

Se ejecutarán distintos escenarios de navegación web en los que se evaluará la capacidad del modelo para detectar contenido sensible en tiempo real, garantizando su eficacia y precisión en la moderación de imágenes.

4. PLAN DE DESARROLLO DEL PROYECTO

4.1. METODOLOGÍA

Para el desarrollo de este proyecto, se ha optado por la metodología en cascada, al considerar que se ajusta adecuadamente tanto a la naturaleza del trabajo como a los objetivos definidos desde el inicio. Esta metodología permite estructurar el proceso en fases bien delimitadas, facilitando una planificación clara y ordenada que se adapta a las necesidades de este tipo de desarrollo.

Al tratarse de un proyecto individual, con recursos limitados y un alcance bien determinado, la elección de un enfoque secuencial resulta especialmente útil para optimizar el tiempo y mantener el control sobre cada etapa. Además, la claridad en los objetivos y la estabilidad del alcance hacen innecesario recurrir a metodologías más iterativas, reservadas habitualmente para trabajos colaborativos y sujetos a cambios frecuentes.

4.2. TECNOLOGÍAS

4.2.1. Tecnologías web y desarrollo en Firefox

Para el desarrollo del proyecto se ha optado por la creación de una extensión para el navegador Firefox, empleando tecnologías web como JavaScript, HTML y CSS. Esta elección responde a la alta compatibilidad, facilidad de despliegue y amplia adopción que ofrece Firefox, lo que permite garantizar una integración sencilla y una experiencia de usuario accesible. Al ser uno de los navegadores más utilizados a nivel global, desarrollar en su entorno asegura un mayor alcance y facilita las pruebas en distintos contextos de uso. Además, el ecosistema de extensiones de Firefox proporciona una estructura bien documentada y herramientas específicas que agilizan el desarrollo.

4.2.2. Modelo de IA

En cuanto al modelo de IA, se ha elegido el uso de CNNs por su eficacia probada en tareas de clasificación de imágenes. Este tipo de modelo destaca por su capacidad para extraer patrones visuales complejos, lo que lo convierte en la opción más adecuada para la detección automática de contenido sensible. Frente a otras arquitecturas más genéricas, las CNNs están específicamente diseñadas para procesar información visual, lo que se traduce en una mayor precisión y rendimiento en este tipo de tareas. [17]

4.2.3. Desarrollo del modelo en Python

El desarrollo de los modelos se realizará en Python, un lenguaje ampliamente utilizado en el ámbito del aprendizaje automático por su versatilidad, legibilidad y la gran cantidad de librerías especializadas que ofrece. Herramientas como *TensorFlow* [18] o *Keras* [19] permiten construir, entrenar y ajustar modelos de forma eficiente, facilitando también su posterior integración con otros componentes del proyecto.

4.2.4. *Dataset* para el entrenamiento

El *dataset* [20]utilizado para el desarrollo del modelo está dividido en cinco categorías: *drawings*, *porn, hentai, neutral* y *sexy*. Se trata de un conjunto de datos en formato ".txt" que contiene enlaces directos (URLs) a las imágenes correspondientes a cada categoría, lo que permite gestionar su descarga de forma automatizada sin necesidad de visualizarlas previamente. Para esta solución concreta se ha optado por trabajar únicamente con las categorías neutral y *sexy*, ya que se ajustan de forma más precisa al enfoque del proyecto y permiten abordar de manera directa la detección de contenido sugerente frente a contenido no sensible. Esta selección estratégica simplifica el modelo y se centra en el tipo de clasificación más relevante para el contexto de la extensión desarrollada.

La decisión de excluir categorías como *porn* o *hentai* responde también a una consideración ética relacionada con el bienestar del alumno desarrollador. Exponer de forma continuada al estudiante a imágenes de contenido explícito podría suponer un impacto psicológico innecesario durante el desarrollo del proyecto, especialmente al tratarse de un trabajo individual y prolongado en el tiempo. De este modo, se busca preservar la salud mental del desarrollador, alineando el enfoque técnico con una perspectiva ética y humana que también forma parte del espíritu de este PFG.

Aun así, cabe destacar que el sistema desarrollado tiene la capacidad de ampliarse fácilmente para incluir nuevas categorías como *porn* o *hentai*, ya contempladas en el propio *dataset* original. Bastaría con habilitar su descarga automática y reentrenar el modelo incluyendo estas clases. Por tanto, aunque el presente proyecto se ha centrado éticamente en las clases *neutral* y *sexy*, su escalabilidad funcional hacia otras categorías más explícitas está garantizada a nivel técnico y permite vislumbrar futuras versiones con un alcance mayor en tareas de moderación automática de imágenes.

La elección de este *dataset* [20] constituye un punto de partida sólido, ya que proporciona de forma completa la clase *neutral* y una base considerable de imágenes correspondientes a la clase *sexy*. Aunque inicialmente existía un desequilibrio entre ambas clases, este fue subsanado mediante la incorporación de imágenes adicionales obtenidas a través de búsquedas automatizadas en bancos públicos como Pixabay [21] y Pexels [22]. Gracias a esta estrategia, se ha conseguido equilibrar el conjunto de datos, alcanzando un total de 10.500 imágenes en cada clase. Esta combinación de fuentes ha permitido construir un *dataset* éticamente gestionado, diverso y suficientemente amplio para el entrenamiento efectivo del modelo.

4.3. Plan de Trabajo

A continuación, se detallan los paquetes de trabajo (PT) definidos en el proyecto, con sus entradas, actividades y salidas principales. Esta estructura permite organizar el desarrollo de forma clara y facilita el seguimiento de cada fase según la planificación temporal establecida.

Tabla 1: Paquete de trabajo 1. Investigación – Elaboración propia

PT 1 – Investigación	**Inicio**	05/02/25
	Fin	24/02/25
Entradas	• Estado del arte sobre moderación de contenido. • Documentación sobre redes neuronales y TensorFlow.js. • Objetivos del proyecto.	
Actividades	• Revisión de soluciones existentes. • Estudio de arquitecturas CNN. • Evaluación de herramientas compatibles con entornos web.	
Salidas	• Justificación del uso de CNNs. • Selección inicial de tecnologías y modelos. • Enfoque técnico definido para el resto del desarrollo.	

Tabla 2: Paquete de trabajo 2. Selección de Datos – Elaboración propia

PT 2 – Selección de Datos	**Inicio**	25/02/25
	Fin	03/03/25
Entradas	• Archivos .txt con URLs clasificados por categoría. • Requisitos del modelo (clases: *neutral* vs *sexy*). • Herramientas de descarga automatizada.	
Actividades	• Implementación de scripts de descarga desde URLs. • Gestión de errores en URLs no válidas. • Refuerzo del *dataset* con imágenes de Pexels y Pixabay.	
Salidas	• *Dataset* estructurado y equilibrado. • Carpeta organizada por clases. • Documentación del proceso de obtención ética de datos.	

Tabla 3: Paquete de trabajo 3. Desarrollo – Elaboración propia

PT 3 – Desarrollo	**Inicio**	04/03/25
	Fin	24/04/25
Entradas	• *Dataset* procesado. • Arquitecturas evaluadas (preentrenadas y propias). • Librerías: TensorFlow, Keras, etc.	
Actividades	• Entrenamiento de múltiples modelos. • Aplicación de data augmentation y técnicas de regularización. • Comparación de métricas y selección del modelo final.	
Salidas	• Modelo final entrenado y validado. • Métricas (*accuracy*, F1, recall...). • Conversión a TensorFlow.js (.json y .bin).	

Tabla 4: Paquete de trabajo 4. Integración – Elaboración propia

PT 4 – Integración del Modelo en la Extensión	**Inicio**	25/04/25
	Fin	21/05/25
Entradas	• Modelo convertido a formato TensorFlow.js. • Estructura inicial de la extensión. • Documentación oficial de TensorFlow.js.	
Actividades	• Carga dinámica del modelo en content.js. • Resolución de incompatibilidades técnicas con TensorFlow.js.	
Salidas	• Extensión funcional con modelo integrado. • Código base final. • Soporte para predicción en tiempo real desde navegador.	

Tabla 5: Paquete de trabajo 5. Pruebas – Elaboración propia

PT 5 – Pruebas	**Inicio**	22/05/25
	Fin	28/05/25
Entradas	• Extensión integrada. • Modelo funcional en entorno web.	
Actividades	• Pruebas en páginas reales. • Evaluación de rendimiento.	
Salidas	• Informe de funcionamiento. • Identificación de limitaciones operativas.	

Tabla 6: Paquete de trabajo 6. Documentación – Elaboración propia

PT 6 – Revisión de Documentación	**Inicio**	29/05/25
	Fin	05/06/25
Entradas	• Borrador de la memoria del PFG. • Resultados finales y métricas. • Comentarios del tutor.	
Actividades	• Inclusión de figuras, tablas y anexos. • Redacción y revisión de capítulos incompletos.	
Salidas	• Memoria final estructurada y corregida.	

A continuación, se presentan los diagramas de Gantt que reflejan gráficamente la planificación descrita. En primer lugar, se muestra una vista general colapsada por paquetes de trabajo (PT), lo que facilita una comprensión rápida de la estructura global del proyecto y de cómo se distribuyen las principales etapas a lo largo del tiempo.

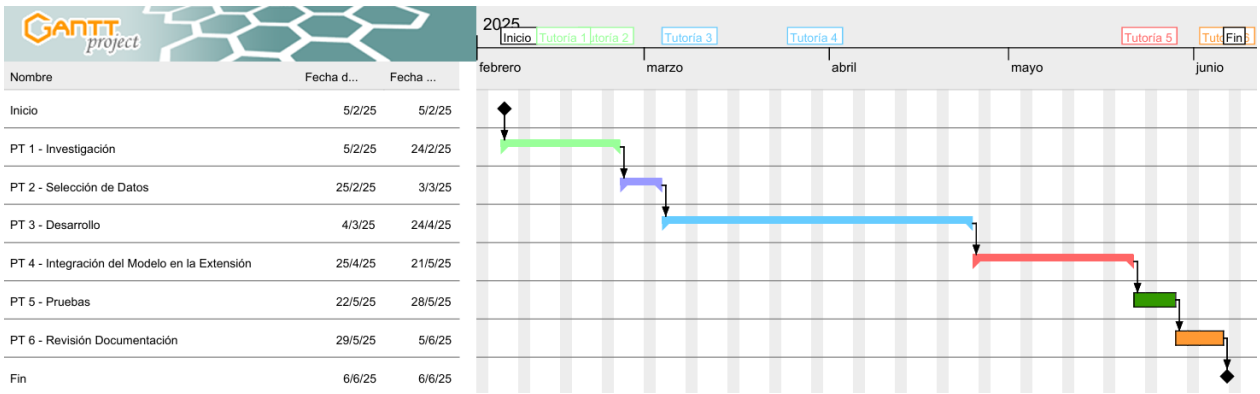

Figura 2: Plan de trabajo general – Elaboración propia.

A continuación, se incluye una versión detallada, donde cada paquete de trabajo se desglosa en sus actividades y tareas correspondientes. Este diagrama ofrece una visión más precisa del cronograma, permitiendo identificar el contenido específico de cada fase, sus fechas previstas y la relación entre componentes.

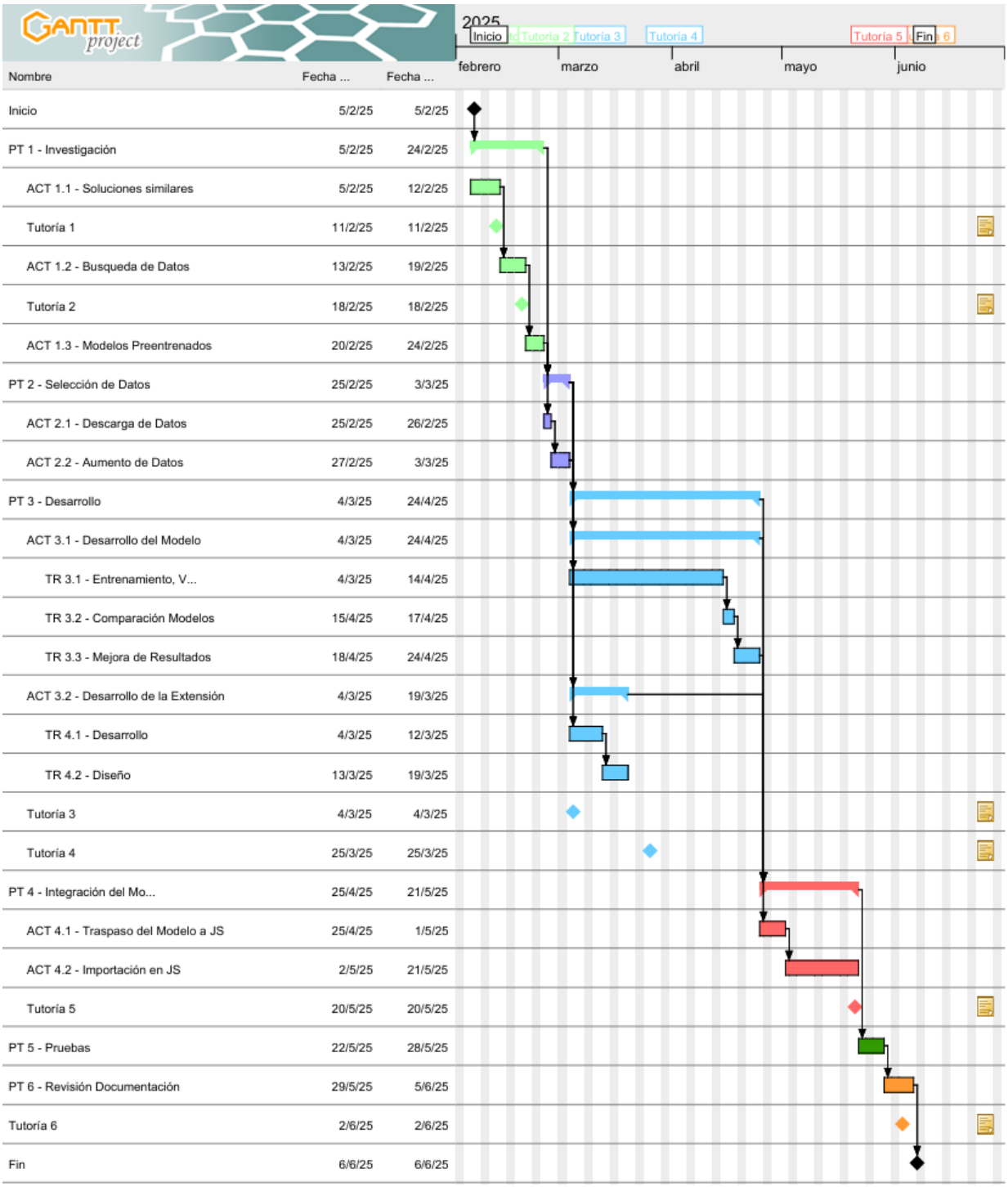

Figura 3: Plan de trabajo detallado – Elaboración propia.

4.4. RECURSOS.

Para llevar a cabo el desarrollo del proyecto de forma eficiente, se han organizado los recursos utilizados en tres categorías diferenciadas: recursos humanos, recursos materiales y recursos de software. Esta clasificación ha permitido una planificación más clara del trabajo y una gestión adecuada de los medios disponibles durante todas las fases del proyecto.

4.4.1. Recursos humanos

El proyecto ha sido desarrollado íntegramente por el propio alumno, quien ha asumido todas las tareas de diseño, implementación, entrenamiento y validación del sistema. Además, ha contado con el acompañamiento y orientación del tutor académico, el profesor Juan Pueyo, cuya supervisión ha sido clave para garantizar el rigor metodológico y técnico del trabajo.

De forma complementaria, se ha recurrido al entorno cercano para contrastar ideas y validar aspectos relacionados con la experiencia de usuario, la funcionalidad de la extensión y su aplicación práctica. Estas contribuciones han resultado muy útiles para adaptar el proyecto a un entorno de uso real.

4.4.2. Recursos materiales

Todo el desarrollo técnico se ha llevado a cabo utilizando un único equipo personal, concretamente un portátil Asus ZenBook, sin necesidad de plataformas en la nube. Este dispositivo ha servido tanto para el entrenamiento local del modelo de red neuronal como para la edición del código fuente de la extensión.

4.4.3. Recursos de software

El entorno de desarrollo principal ha sido VS Code, que ha permitido integrar de forma ágil la programación en Python (para el modelo) y en JavaScript, HTML y CSS (para la extensión del navegador).

El proyecto se ha apoyado en un conjunto de herramientas y librerías de software libre y de código abierto, entre las que destacan Python 3, TensorFlow, Keras, NumPy, Pandas, Matplotlib y scikit-learn, fundamentales para la construcción, entrenamiento y evaluación del modelo.

También se ha empleado Git como sistema de control de versiones, lo que ha permitido organizar el desarrollo de forma estructurada y mantener un registro detallado y seguro de los cambios introducidos a lo largo del proyecto.

4.5. Costes

A continuación, se presenta el resumen económico del proyecto, dividido entre gastos operativos (OPEX) y de capital (CAPEX), conforme a las directrices de estimación presupuestaria habituales en el sector del desarrollo software. La tabla detalla el coste directo de cada categoría, el margen aplicado para garantizar la viabilidad del proyecto, los impuestos correspondientes y el coste total resultante.

Tabla 7: Costes globales – Elaboración propia

Categoría	Descripción	Tipo	Coste	Margen	Impuestos	Total
Personal	Trabajadores del proyecto	OPEX	7.270 *1,34 = 9.741,8€	3.896,72€ (+40%)	2.864,09€ (21%)	16.502,61€
Licencias Suscripción	Software de suscripción	OPEX	0,0	0,0	0,0	0,0
Infraestructuras	PaaS, Hosting, Cloud fees	OPEX	0,0	0,0	0,0	0,0
Suministros	Electricidad, telefonía, cuota oficina	OPEX	100,0€	30,0€ (+30%)	27,30€ (+21%)	157,30€
Varios	Documentación, Viajes, abogados	OPEX	150,0€	45,0€ (+30%)	19,50€ (+21%)	214,50€
Hardware	Equipos comprados para el proyecto	CAPEX	0,0	0,0	0,0	0,0
Licencias Adquiridas	Software adquirido para el proyecto	CAPEX	0,0	0,0	0,0	0,0
TOTAL OPEX						**16.874,41€**
TOTAL CAPEX						**0.0**
TOTAL						**16.874,41€**

La siguiente tabla desglosa los costes asociados a la categoría de personal, considerando los distintos roles asumidos a lo largo del desarrollo del proyecto. Para cada perfil se indican las horas estimadas de dedicación, el coste neto por hora, y el coste total asociado a su participación. Esta división permite visibilizar las distintas competencias implicadas y valorar de forma detallada el esfuerzo invertido en cada área del proyecto.

Tabla 8: Costes de Personal – Elaboración propia.

Perfil	Horas	Coste neto* por hora	Coste neto* total
Investigador IA	60	18€	1.080€
Desarrollador CNN	90	25€	2.250€
Desarrollador Extensión	50	20€	1.000€
Integrador de modelos	50	22€	1.100€
Tester	20	20€	400€
Documentación y redacción	40	16€	640€
Jefe de proyecto	20	40€	800€
TOTAL	**330**		**7.270€**

4.6. CONDICIONANTES Y LIMITACIONES

A lo largo del desarrollo del proyecto, se han identificado diversos condicionantes y limitaciones que han afectado tanto a la planificación inicial como a la ejecución técnica del mismo.

Inicialmente, se planteó desarrollar un sistema de clasificación de imágenes pornográficas mediante técnicas de aprendizaje automático. No obstante, surgieron dificultades éticas importantes relacionadas con la creación o validación del *dataset* necesario, ya que no existía una forma viable de comprobar su contenido sin exponerse directamente a imágenes explícitas, lo cual comprometía el bienestar del alumno. Por este motivo, el anteproyecto se reformuló en una primera etapa hacia la clasificación de imágenes de camisetas, concretamente prendas deportivas como camisetas de fútbol o entrenamiento.

Ante esta situación, y con el objetivo de orientar el trabajo hacia una aplicación más útil y respetuosa, se decidió redirigir el proyecto hacia la clasificación de imágenes en tres niveles: normal, semidesnudos y desnudos. Finalmente, se optó por centrarse específicamente en la detección de semidesnudos, concretamente en personas en bañador. Este cambio permitió encontrar y utilizar un *dataset* [20] ya existente, lo que facilitó significativamente el desarrollo técnico del proyecto.

En cuanto a la versión de prueba de la extensión, surgieron varias dificultades significativas. Una de ellas fue la necesidad de añadir un *checkbox* individual a cada imagen para poder mostrar su porcentaje de clasificación. El principal inconveniente radicó en que las imágenes procedían directamente de búsquedas en Google, cuya página no permite controlar eventos de clic desde extensiones del navegador, impidiendo así esta funcionalidad.

Otra limitación importante estuvo relacionada con la obtención y preparación del *dataset*. Se presentaron múltiples problemas en la descarga automática de imágenes desde URLs proporcionadas, resultando en una reducción del volumen inicial de datos. Para solventar esta limitación, fue necesario complementar el *dataset* original mediante la descarga automática de imágenes adicionales desde plataformas como Pixabay [21] y Pexels [22], lo que requirió trabajo adicional en la gestión y procesamiento de estas nuevas fuentes.

Adicionalmente, surgieron problemas técnicos derivados de la incompatibilidad de la versión *Manifest V3* de Google Chrome con la integración del modelo en TensorFlow.js. Esta incompatibilidad está relacionada con que TensorFlow.js aún no ha sido adaptado completamente para su correcto funcionamiento en la versión *Manifest V3*. Aunque medios oficiales de Google tiene publicados métodos para su integración [23], las nuevas políticas de privacidad de Chrome restringen notablemente el tratamiento de imágenes respecto a la versión anterior (*Manifest V2*). Como solución alternativa, finalmente se decidió implementar la extensión para el navegador Firefox, el cual todavía soporta la versión *Manifest V2* y mantiene políticas de privacidad más flexibles respecto al procesamiento de imágenes.

Conviene señalar además que, pese a que TensorFlow.js proporciona ejemplos oficiales [24] para integrar modelos en extensiones de Chrome, en la práctica dichos ejemplos no

funcionan correctamente bajo *Manifest V3*. Muchas de las funciones sugeridas no son compatibles con los entornos de ejecución actuales, lo que genera errores que no están debidamente documentados ni resueltos en los repositorios oficiales. Esta falta de soporte efectivo por parte de la propia documentación oficial dificultó considerablemente la integración inicial, y terminó por confirmar la necesidad de migrar el desarrollo a Firefox, donde el entorno resulta más estable y permisivo para este tipo de soluciones.

Asimismo, la última versión disponible de TensorFlow no es compatible con Python 3.12. Este inconveniente obligó a utilizar versiones anteriores de Python, Keras y NumPy para entrenar adecuadamente el modelo y asegurar la compatibilidad con la versión más reciente del conversor a TensorFlow.js [25].

Todos estos condicionantes y limitaciones supusieron un considerable esfuerzo adicional en mitad del desarrollo del Proyecto Final de Grado, afectando tanto a la planificación como al cumplimiento del plazo de entrega al tutor. Por un lado, representaron una dificultad técnica significativa, especialmente por la escasa utilidad de la documentación oficial de TensorFlow.js [24]. Sin embargo, también han supuesto una oportunidad de aprendizaje real, ya que han permitido desarrollar competencias de resolución de problemas en un contexto complejo y cambiante. Afortunadamente, estas incidencias surgieron con suficiente antelación, lo que ha permitido reorientar el desarrollo hacia Firefox y ajustar el entorno de trabajo.

5. DESARROLLO DE LA SOLUCIÓN TÉCNICA

A continuación, se expone de forma detallada el desarrollo de cada paquete de trabajo contemplado en la planificación del proyecto, describiendo las actividades realizadas, los resultados alcanzados, las incidencias encontradas y, en caso de haberlas, las desviaciones respecto a lo previsto.

5.1. INVESTIGACIÓN

El paquete de trabajo correspondiente a la fase de investigación se ha llevado a cabo durante las primeras semanas del proyecto. Su ejecución ha seguido lo previsto en la planificación inicial, y ha permitido sentar las bases teóricas y técnicas del desarrollo posterior.

Durante esta fase se ha realizado una búsqueda intensiva de soluciones existentes relacionadas con la detección de contenido sensible en imágenes, centrándose en herramientas ya implementadas como extensiones de navegador, APIs comerciales y modelos *open-source*. Esta labor ha permitido identificar las limitaciones de muchas de las propuestas actuales, basadas en listas negras o categorizaciones genéricas.

En paralelo, se ha llevado a cabo una investigación técnica de los tipos de inteligencia artificial más adecuados para la tarea propuesta. Se ha concluido que el uso de CNNs ofrece el mejor equilibrio entre precisión, eficiencia computacional y facilidad de implementación. Se han comparado distintas arquitecturas (como MobileNetV2 [26], EfficientNetB3 o VGG16) evaluando sus características y compatibilidad con TensorFlow.js para comprobar la viabilidad del proyecto. [27]

Como resultado, esta fase ha permitido definir con claridad el enfoque técnico del proyecto, seleccionar las tecnologías base que se emplearán en el desarrollo —concretamente CNNs y TensorFlow.js—, así como establecer los criterios fundamentales que se utilizarán posteriormente para la evaluación y validación de los modelos entrenados.

Entre las dificultades encontradas, destaca la abundancia de soluciones comerciales poco transparentes, lo que ha dificultado el análisis detallado de su funcionamiento interno. Además, se ha comprobado que no todos los modelos preentrenados son fácilmente adaptables al formato requerido para su integración en navegador, lo que ha condicionado la elección del modelo base.

5.2. SELECCIÓN DE DATOS

El proceso de selección y recopilación de datos ha constituido una fase fundamental para el desarrollo del modelo de clasificación. Dado que el rendimiento de las redes neuronales depende en gran medida de la calidad y representatividad del conjunto de entrenamiento, se ha prestado especial atención tanto al origen como a la organización ética y estructurada de las imágenes empleadas.

Inicialmente, se ha optado por un enfoque automatizado para la descarga de imágenes a partir de archivos de texto que contenían URLs [20] previamente clasificadas por categorías. Mediante un programa desarrollado en Python, se han procesado dichas listas, consultando una a una las direcciones y descargando las imágenes válidas en carpetas específicas organizadas por clase. Esta metodología ha permitido mantener un control total sobre el contenido descargado, garantizando que todo el proceso se realizara de forma ética, reproducible y sin recurrir a repositorios cuestionables.

No obstante, durante la ejecución de este procedimiento se han detectado importantes dificultades. Una proporción considerable de las URLs incluidas en los archivos ya no estaba disponible o conducía a imágenes eliminadas o inaccesibles, lo que ha limitado significativamente el volumen de datos útiles, especialmente en la categoría "*sexy*".

Para mitigar esta limitación, se ha implementado un segundo método complementario basado en la obtención de imágenes desde bancos públicos con licencia libre como Pixabay [21] y Pexels [22]. A través de sus respectivas APIs, se ha desarrollado un script capaz de realizar peticiones automáticas introduciendo términos de búsqueda concretos, y almacenar las imágenes recuperadas en directorios clasificados. Esta estrategia ha resultado particularmente útil para reforzar la clase minoritaria y aumentar su diversidad visual sin comprometer la integridad ética del alumno.

Como resultado, se ha conseguido conformar un conjunto de datos estructurado en dos clases bien diferenciadas: imágenes neutrales e imágenes de contenido sexualmente sugerente.

El *dataset* se ha sometido a un preprocesamiento de las imágenes para garantizar su homogeneidad y adecuación al modelo. En todos los casos se ha comprobado que las imágenes estuvieran en formato RGB, se han redimensionado a un tamaño uniforme de píxeles y se han normalizado dividiendo sus valores de píxeles por 255.0. Tras este proceso, el *dataset* ha servido de base para el posterior entrenamiento y validación del modelo de clasificación convolucional.

Se ha realizado un collage que representa visualmente una muestra del conjunto de imágenes que componen el *dataset* empleado. Cada collage recoge ejemplos reales de ambas clases utilizadas en el entrenamiento: *neutral* y *sexy*. Con el fin de preservar la naturaleza ética del proyecto y evitar cualquier contenido explícito, el collage correspondiente a la clase *sexy* ha sido difuminado levemente. Esta medida permite intuir el tipo de imágenes incluidas sin mostrar detalles explícitos. Para consultar los collages completos de ambas clases, véase en la Figura 20 y Figura 21.

5.3. DESARROLLO

5.3.1. Desarrollo del modelo

5.3.1.1. Explicación de técnicas utilizadas

Durante esta fase del proyecto se ha llevado a cabo un análisis comparativo y experimental de distintas arquitecturas de CNN, tanto diseñadas desde cero por el propio alumno como basadas en modelos preentrenados disponibles en la biblioteca Keras [26].

En el caso de los modelos preentrenados, se ha realizado una selección fundamentada a partir del análisis de las estadísticas oficiales proporcionadas por Keras [27](precisión, número de parámetros, tamaño y tiempo de inferencia por paso). Tras esta revisión, se ha optado por emplear MobileNetV2 y EfficientNetB3 como puntos de partida, al considerar que ofrecen un equilibrio adecuado entre rendimiento y eficiencia computacional para la tarea de clasificación binaria planteada.

En paralelo, se han construido varias arquitecturas personalizadas, modificando variables como el número de capas convolucionales (entre 3 y 4), la cantidad de filtros en cada capa, así como la dimensión de las capas densas. Estos modelos se han utilizado para establecer una línea base y experimentar con diferentes combinaciones de hiperparámetros, con el fin de valorar su rendimiento en relación con los modelos preentrenados.

Los principales hiperparámetros ajustados a lo largo del proceso han sido los siguientes:

- *BATCH_SIZE*: tamaño de los lotes de imágenes que se procesan simultáneamente durante cada iteración del entrenamiento. Determina el equilibrio entre velocidad y estabilidad de convergencia.
- *IMG_SIZE*: resolución en píxeles a la que se han redimensionado todas las imágenes de entrada, con el fin de unificar el formato y reducir la carga computacional.
- *DROPOUT*: técnica de regularización que consiste en desactivar aleatoriamente un porcentaje de neuronas durante el entrenamiento, con el objetivo de evitar el sobreajuste.
- *LEARNING_RATE*: tasa de aprendizaje inicial del optimizador, que controla el tamaño de los pasos que da el modelo en cada actualización de pesos.
- *FactorLR*: factor por el que se reduce la tasa de aprendizaje cuando no se observa mejora en la función de pérdida de validación, permitiendo una convergencia más fina en fases avanzadas del entrenamiento.
- *EPOCHS*: número máximo de iteraciones completas sobre el conjunto de entrenamiento.

- *Patience*: número de épocas consecutivas sin mejora en la validación tras las cuales se detiene anticipadamente el entrenamiento, evitando así un entrenamiento innecesario.

En todos los casos, se ha utilizado el optimizador Adam, ampliamente reconocido por su rendimiento eficiente en tareas de clasificación, junto con la función de pérdida '*binary_crossentropy*', adecuada para problemas binarios. Asimismo, se ha empleado RELU como función de activación en las capas intermedias y sigmoide en la capa de salida, dado que se trata de una tarea de clasificación binaria, donde esta configuración permite una mejor convergencia y una salida interpretada como probabilidad.

Previo al proceso de entrenamiento, el conjunto de datos se ha dividido en tres subconjuntos con la siguiente proporción: 70 % para entrenamiento (14.700 imágenes), 20 % para validación (4.200 imágenes) y 10 % para test (2.100 imágenes). Esta distribución se ha preferido frente al esquema más habitual del 80 %-10 %-10 % o 80 %-20 %, dado el elevado volumen total de imágenes disponibles. Se ha considerado que un 10 % representa una muestra suficientemente representativa para la evaluación final del modelo, lo que ha permitido destinar una mayor proporción de datos al entrenamiento, con el objetivo de mejorar la capacidad de generalización y obtener un mayor rendimiento en el modelo final.

El entrenamiento se ha complementado con un sistema de generación dinámica de imágenes basado en la técnica de *data augmentation*. Para ello, se han utilizado transformaciones aleatorias sobre las imágenes de entrenamiento con el fin de aumentar la variabilidad del conjunto y mejorar la capacidad de generalización del modelo. Las transformaciones incluyeron rotaciones, desplazamientos, zoom y volteo horizontal:

Tabla 9: Parámetros data augmentation – Elaboración propia.

Parámetro	Valor
rotation_range	15
width_shift_range	0.1
height_shift_range	0.1
zoom_range	0.1
horizontal_flip	*True*

Además, se han incorporado dos técnicas adicionales para optimizar el entrenamiento:

- *ReduceLROnPlateau*: monitoriza la función de pérdida de validación y reduce automáticamente el LR si este deja de mejorar tras varias épocas consecutivas.

Tabla 10: Parámetros reducción de LR – Elaboración propia.

Parámetro	Valor
monitor	'val_loss'
factor	FarctorLR
patience	2
min_lr	0.0001
verbose	1

- *EarlyStopping*: detiene el entrenamiento anticipadamente si no se detectan mejoras tras un número determinado de épocas, restaurando los pesos correspondientes a la mejor iteración validada.

Tabla 11: Parámetros de parada temprana – Elaboración propia.

Parámetro	Valor
Monitor	'val_loss'
Patience	2
Restore_best_weights	True

Esta combinación de técnicas ha permitido ajustar el entrenamiento de forma eficiente, evitando tanto el sobreentrenamiento como la pérdida de tiempo computacional innecesaria.

5.3.1.2. Evaluación comparativa y elección del modelo final

Una vez definidas las distintas arquitecturas y configuraciones, se ha procedido a entrenar y evaluar tanto modelos preentrenados como modelos diseñados por el propio alumno, con el objetivo de seleccionar la solución más adecuada para el despliegue final del sistema.

Los resultados obtenidos por los modelos preentrenados han sido muy satisfactorios en términos de precisión y generalización. En particular, el modelo MobileNetV2 ha demostrado un rendimiento notable, alcanzando una precisión en test del 95,23 %, con valores de precision, recall y F1-score superiores al 94 %, y una pérdida de tan solo 12,36 **%** en el conjunto de prueba. Estos resultados lo sitúan como la mejor arquitectura evaluada en cuanto a métricas puras.

No obstante, es importante destacar que MobileNetV2, si bien ha ofrecido el mejor desempeño, presenta un número considerablemente mayor de parámetros en comparación con los modelos diseñados desde cero. Esto se debe a que está concebido como una red genérica y profunda, capaz de clasificar una gran variedad de objetos en tareas multiclase, lo que repercute en un mayor tamaño, complejidad y carga computacional. Para un caso específico como el presente, centrado únicamente en una clasificación binaria, este nivel de complejidad resulta innecesario.

Por el contrario, el modelo personalizado creado por el alumno ha logrado unos resultados muy cercanos a los del modelo preentrenado, con una precisión en test del 94,05 %, una F1-score de 94,03 % y un comportamiento muy equilibrado en cuanto a precisión y recall. Todo ello con una arquitectura mucho más ligera, compuesta por apenas 392.481 parámetros (más adelante), frente a los 2.340.033 de parámetros de MobileNetV2. Esta diferencia supone una ventaja significativa en términos de eficiencia, especialmente al considerar que el modelo debe cargarse y ejecutarse desde el navegador.

En consecuencia, y tras valorar el equilibrio entre rendimiento y ligereza, se ha optado por implementar como solución final el modelo propio desarrollado desde cero. Este modelo ofrece un rendimiento excelente, con una precisión superior al 94 % en todos los conjuntos (entrenamiento, validación y prueba), y presenta una estructura óptima para su integración en entornos web ligeros.

El modelo final se compone de cuatro bloques de convolución con reducción de tamaño (mediante *max pooling*), seguidos de una capa que transforma los datos en un vector (*flatten*), una capa totalmente conectada de 64 unidades, una capa de abandono (*dropout*) para prevenir sobreajuste, y una capa final con activación sigmoide, encargada de emitir la predicción binaria.

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv1 (Conv2D)               (None, 126, 126, 16)      448
_________________________________________________________________
pool1 (MaxPooling2D)         (None, 63, 63, 16)        0
_________________________________________________________________
conv2 (Conv2D)               (None, 61, 61, 32)        4640
_________________________________________________________________
pool2 (MaxPooling2D)         (None, 30, 30, 32)        0
_________________________________________________________________
conv3 (Conv2D)               (None, 28, 28, 64)        18496
_________________________________________________________________
pool3 (MaxPooling2D)         (None, 14, 14, 64)        0
_________________________________________________________________
conv4 (Conv2D)               (None, 12, 12, 128)       73856
_________________________________________________________________
pool4 (MaxPooling2D)         (None, 6, 6, 128)         0
_________________________________________________________________
flatten (Flatten)            (None, 4608)              0
_________________________________________________________________
dense1 (Dense)               (None, 64)                294976
_________________________________________________________________
dropout (Dropout)            (None, 64)                0
_________________________________________________________________
output (Dense)               (None, 1)                 65
=================================================================
Total params: 392,481
Trainable params: 392,481
Non-trainable params: 0
```

Figura 4: Arquitectura del modelo – Elaboración propia.

Los hiperparámetros que han obtenido los mejores resultados han sido los siguientes:

Tabla 12: Hiperparámetros del modelo – Elaboración propia

Hiperparámetro	Valor
BATCH_SIZE	8
IMG_SIZE	128
DROPOUT	0.3
LEARNING_RATE	0.0008
FactorLR	0.7
EPOCHS	40
PatienceES	6
F_ACT	Relu
F_ACT_2	Sigmoid

A continuación, se presentan los resultados obtenidos durante la fase de entrenamiento, validación y prueba del modelo final, incluyendo las principales métricas de evaluación, la evolución de la precisión y la pérdida a lo largo del entrenamiento, así como la matriz de confusión que resume su comportamiento sobre el conjunto de test.

Las siguientes métricas permiten valorar de forma global el rendimiento del modelo:

- ***Accuracy***: Porcentaje de predicciones correctas sobre el total de muestras evaluadas.
- ***Loss***: Medida del error global del modelo durante la predicción; valores más bajos indican un mejor ajuste a los datos reales.
- ***Precision***: Proporción de imágenes clasificadas como "sensibles" que efectivamente lo eran.
- ***Recall***: Proporción de imágenes realmente "sensibles" que el modelo ha detectado correctamente.
- ***F1-Score***: Media armónica entre *precision* y *recall*, que ofrece una visión equilibrada del rendimiento cuando ambas métricas son relevantes.

Las métricas obtenidas en el conjunto de prueba reflejan un rendimiento muy sólido y equilibrado, con un nivel de acierto elevado y una distribución de errores sin sesgos significativos. Se resumen a continuación:

Tabla 13: Valores de las métricas – Elaboración propia.

Métrica	Valor
Accuracy	94.05%
Loss	17.63%
Precision	94.34%
Recall	93.71%
F1-Score	93.03%

La matriz de confusión correspondiente al conjunto de test permite examinar de manera más específica los aciertos y errores por clase. El modelo ha clasificado correctamente 991 imágenes neutrales y 984 imágenes de contenido sensible (*sexy*), lo que evidencia una elevada capacidad de discriminación entre ambas categorías. No obstante, se han registrado 59 falsos positivos (imágenes neutras clasificadas como "*sexy*") y 66 falsos negativos (imágenes "*sexy*" clasificadas como neutras), lo cual es coherente con los valores obtenidos de *precision* y *recall*, y confirma un comportamiento equilibrado del modelo en ambos tipos de error. Ver matriz de confusión en Figura 24.

En conjunto, los datos evidencian que el modelo presenta un comportamiento equilibrado y eficaz, sin sesgos marcados hacia ninguna de las dos clases. En las siguientes gráficas se

muestra la evolución de la precisión y la pérdida durante el entrenamiento y la validación. Puede observarse una mejora progresiva sin signos de sobreajuste significativo:

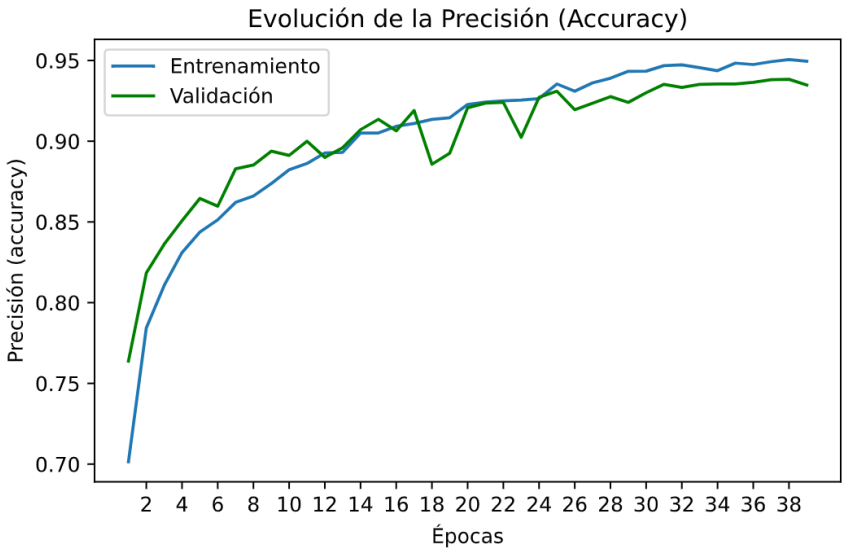

Figura 5: Evolución de la precisión – Elaboración propia.

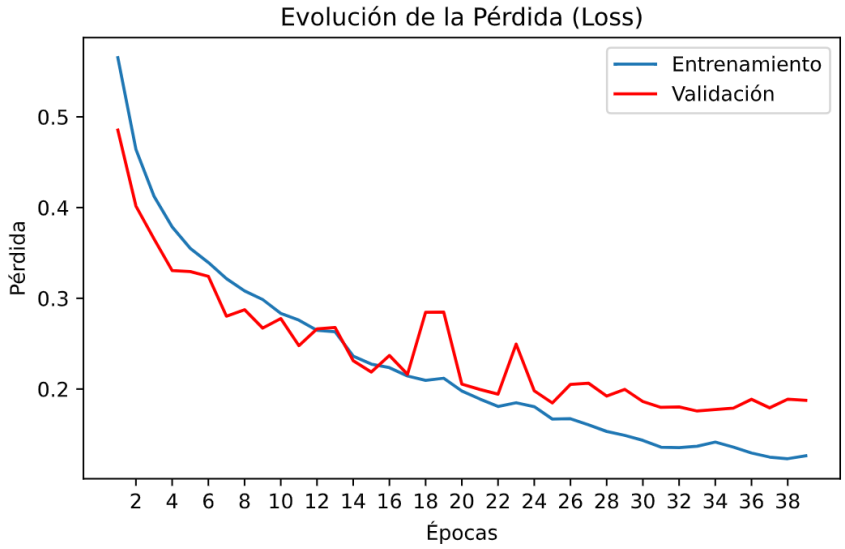

Figura 6: Evolución de la pérdida – Elaboración propia.

A lo largo del entrenamiento, el LR ha ido disminuyendo progresivamente, como ya se ha explicado anteriormente gracias al uso de la técnica ReduceLROnPlateau. Esta estrategia ha permitido facilitar una convergencia más rápida en las primeras fases del entrenamiento y una optimización más precisa en las etapas finales. La gráfica correspondiente puede consultarse en Figura 25.

Adicionalmente, se ha evaluado el modelo mediante un procedimiento de validación cruzada de cinco particiones (K=5) con el objetivo de comprobar la estabilidad y capacidad de generalización del mismo en distintas divisiones del conjunto de datos. Los resultados obtenidos han sido muy consistentes en todas las iteraciones, con una *accuracy* media del

94,78 %, una F1-score media del 93,18 % y desviaciones inferiores al 0,5 % en la mayoría de las métricas. La pérdida de validación se ha mantenido estable, con una media de 18,35 %, mientras que la pérdida media en entrenamiento ha sido de 59,31 %, un valor que, aunque elevado, es esperable al utilizar la función *binary_crossentropy*, ya que esta penaliza más intensamente los errores con alta confianza. Este comportamiento no indica un mal rendimiento, como demuestran el resto de métricas obtenidas. En conjunto, los resultados confirman que el modelo es robusto, generaliza bien y mantiene un rendimiento fiable independientemente de la partición utilizada.

Tabla 14: Valores de métricas K-Fold – Elaboración propia.

Métrica	Media (%)	Desviación (%)
Accuracy	94.78	0.37
Loss	59.31	0.61
Val_Accuracy	93.18	0.48
Val_Loss	18.35	1.58
Precision	93.19	0.41
Recall	93.16	0.64
F1-Score	93.18	0.49

En definitiva, el modelo desarrollado ha demostrado ser una solución eficaz, precisa y ligera para la tarea de clasificación binaria, con un rendimiento robusto y estable en todas las evaluaciones realizadas. Aunque durante el desarrollo se presentaron algunas dificultades técnicas, como la imposibilidad de completar entrenamientos largos en Google Colab por limitaciones de tiempo de sesión, lo que obligó a continuar el proceso en un equipo local —afectando al rendimiento general del sistema durante las fases de entrenamiento—, el modelo ha alcanzado un nivel de desempeño notable y se considera plenamente adecuado para su integración en la solución final del proyecto.

5.3.2. Desarrollo de la extensión

El desarrollo de la extensión del navegador se ha realizado de forma progresiva y modular, avanzando en paralelo al entrenamiento del modelo de clasificación. Esta estrategia ha permitido trabajar desde fases tempranas en la construcción de la arquitectura general de la herramienta, validando su funcionamiento en entorno real aunque el modelo definitivo aún no estuviera disponible.

En las primeras etapas, con el objetivo de probar el flujo de ejecución, la detección de imágenes en páginas web, la superposición visual de resultados y la experiencia de usuario, se implementó una versión preliminar que asignaba a cada imagen un porcentaje aleatorio de probabilidad. Esta simulación del comportamiento del modelo resultó clave para verificar la lógica de funcionamiento de la extensión sin depender todavía del modelo entrenado, y permitió iterar rápidamente en aspectos como el diseño visual, la estructura de control y la manipulación dinámica del DOM.

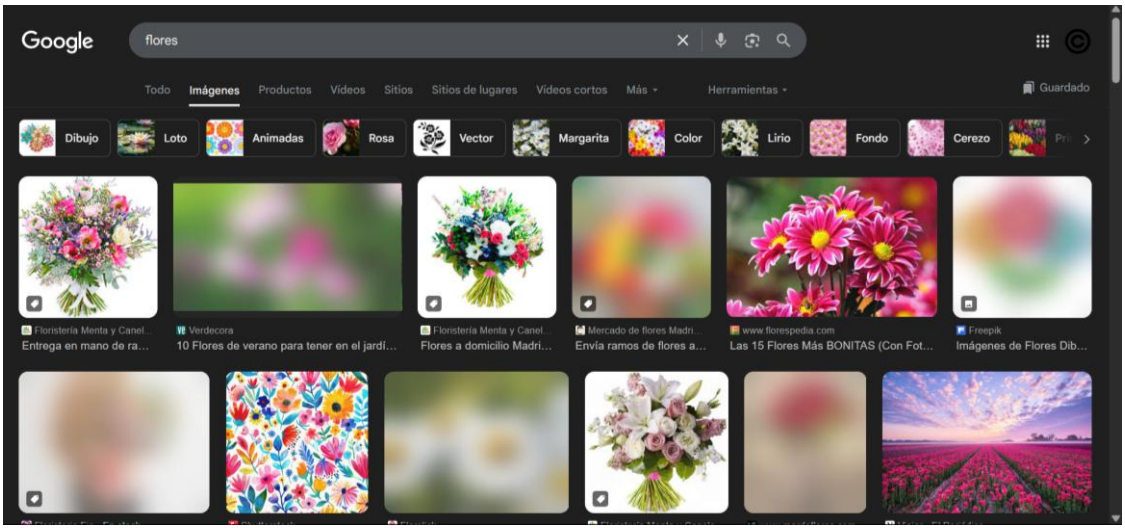

Figura 7: Versión beta de la extensión – Elaboración propia.

Durante esta fase también se definió la estructura base de la extensión, compuesta por los archivos *manifest.json* (configuración y permisos), popup.html y popup.js (interfaz gráfica de usuario), y content.js (script principal ejecutado sobre las páginas web). Se diseñó una interfaz clara y funcional accesible desde el icono del navegador, que permite activar o desactivar la extensión y visualizar en tiempo real el número de imágenes analizadas y protegidas. Para ver con mayor detalle el diseño de los iconos y logos: Figura 22 y Figura 23.

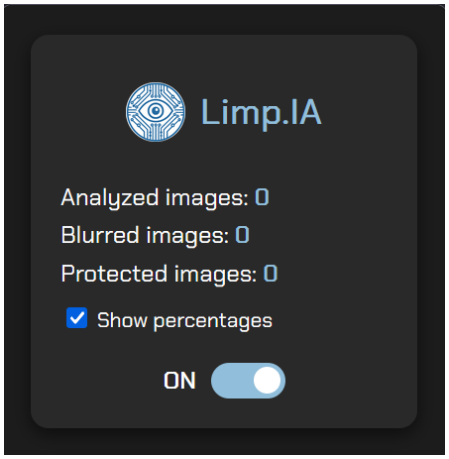

Figura 8: Popup de la extensión – Elaboración propia.

Además, se implementaron mecanismos de detección automática de nuevas imágenes mediante un observador de mutaciones en el DOM, garantizando que los contenidos cargados dinámicamente también sean interceptados y procesados por la extensión.

Uno de los principales problemas encontrados durante esta etapa fue la necesidad de trasladar el desarrollo desde Google Chrome a Mozilla Firefox, debido a restricciones y dificultades técnicas relacionadas con la integración del modelo en el entorno de ejecución de Chrome. Este cambio implicó además una adaptación de la estructura de la extensión, pasando de la versión de manifiesto V3 (utilizada en Chrome) a la versión V2 compatible con Firefox, lo que afectó tanto a la gestión de permisos como a la ejecución de scripts. Este aspecto se detalla con mayor profundidad en el apartado dedicado a la integración del modelo.

5.4. Integración del Modelo en la Extensión

La integración del modelo entrenado en la extensión del navegador representaba, a priori, una de las fases más sencillas del proyecto, dado que la documentación oficial de TensorFlow.js proporciona ejemplos específicos sobre cómo realizar esta tarea [28]. Sin embargo, durante el desarrollo surgieron una serie de limitaciones técnicas y de compatibilidad que convirtieron esta etapa en una de las más complejas del proceso.

En primer lugar, la mayoría de los ejemplos y recursos disponibles en la documentación oficial están orientados a extensiones desarrolladas bajo M*anifest V3*, el estándar actual para Google Chrome. No obstante, las políticas de seguridad y privacidad introducidas en esta versión imponen restricciones severas al tratamiento de imágenes, especialmente en lo relativo al acceso y análisis de contenido visual mediante scripts. Como consecuencia, el uso de modelos cargados con TensorFlow.js queda prácticamente inhabilitado en entornos de ejecución tipo *content scrip*t, que son necesarios para analizar imágenes directamente en las páginas web.

Ante esta limitación, se optó por adaptar la extensión al navegador Mozilla Firefox, que mantiene compatibilidad con M*anifest V2*, una versión más flexible y permisiva respecto al tratamiento de contenido multimedia. Este cambio implicó reestructurar parte del desarrollo inicial, pero resultó fundamental para poder cargar el modelo y aplicar predicciones sobre las imágenes detectadas.

Además, se identificaron problemas adicionales derivados de incompatibilidades entre las versiones más recientes de Python (como la 3.12), TensorFlow y NumPy. Concretamente, los cambios introducidos en la gestión interna de vectores por parte de NumPy provocaban errores de ejecución al trabajar con TensorFlow, dificultando el proceso de conversión del modelo al formato requerido por TensorFlow.js. Para solventarlo, se decidió retroceder a una versión estable de Python (3.7) y utilizar una versión anterior de NumPy compatible con la última versión funcional de TensorFlow. Esta decisión no ha afectado ni a la arquitectura del modelo ni a su rendimiento, pero ha sido clave para completar con éxito el proceso de conversión.

Una vez resueltos estos inconvenientes, se procedió a la conversión del modelo entrenado al formato de TensorFlow.js, generando los archivos necesarios: un archivo JSON que define la estructura de la red y, en este caso, un archivo BIN que contiene los pesos del modelo. Estos ficheros se incorporan a la extensión y se cargan dinámicamente en el navegador al iniciar la ejecución, permitiendo la clasificación de imágenes directamente en el entorno web.

La versión final, desarrollada bajo M*anifest V2* y ejecutada en Firefox, permite analizar las imágenes presentes en cualquier página, con excepción de aquellas bloqueadas por políticas CORS o restricciones de privacidad del propio navegador, que son gestionadas mediante mensajes informativos al usuario.

5.5. PRUEBAS

El sistema funciona correctamente en la gran mayoría de los casos, procesando las imágenes presentes en las páginas web y aplicando el difuminado cuando corresponde. Cuando una imagen es procesada con éxito, el modelo le asigna una predicción en forma de porcentaje que indica la probabilidad de que su contenido sea sensible. Por defecto, este porcentaje no se muestra. Si el usuario ha activado esta opción en la configuración de la extensión, el porcentaje se mostrará superpuesto sobre las imágenes. En cualquier caso, si la probabilidad supera el 50 %, la imagen se difumina automáticamente, aplicando un filtro visual que impide su visualización directa.

De manera puntual, algunas imágenes no pueden ser procesadas debido a políticas de privacidad o restricciones CORS, que impiden el acceso al contenido desde scripts de extensión. En estos casos, el sistema informa al usuario mediante mensajes visibles o iconos de advertencia, de modo que sea consciente de que ciertas imágenes no han podido ser evaluadas automáticamente. Asimismo, si por cualquier otro motivo una imagen no puede ser analizada correctamente —por ejemplo, por errores de red, problemas en el procesamiento o formatos incompatibles—, el sistema también lo notifica, garantizando así la transparencia del proceso. Estos casos, junto con ejemplos visuales se recogen en el Anexo D: Pruebas

5.6. REVISIÓN DE LA DOCUMENTACIÓN

Durante todo el proceso de desarrollo del proyecto, se ha mantenido una documentación técnica actualizada que recoge cada una de las decisiones, implementaciones y ajustes realizados en las distintas fases. Esta documentación ha sido revisada de forma continua para asegurar su coherencia con el estado real del proyecto y facilitar la trazabilidad de los cambios introducidos.

En la fase final, se ha llevado a cabo una revisión exhaustiva y estructural de todos los documentos generados, con especial atención a la consistencia terminológica empleada a lo largo de la memoria y a la homogeneidad en el estilo de redacción. También se han actualizado todos los esquemas, diagramas y fragmentos de código para que reflejen con exactitud la solución implementada finalmente. Asimismo, se ha verificado la veracidad de los resultados y métricas presentados, contrastándolos con los datos obtenidos durante las pruebas prácticas. Por último, se ha cuidado la organización y presentación de los contenidos para asegurar una lectura clara y comprensible, tanto desde un enfoque técnico como divulgativo.

Asimismo, se ha garantizado que todos los anexos, tablas de resultados, capturas de pantalla, hipervínculos y referencias a recursos externos estén correctamente enlazados y sean accesibles. Esta labor ha sido esencial para consolidar la memoria como un reflejo fiel, riguroso y profesional del trabajo realizado, asegurando su valor como documento final de entrega.

6. RESULTADOS

En este capítulo se presentan los resultados obtenidos durante el desarrollo del proyecto, analizando el grado de cumplimiento de los objetivos específicos y su correspondencia con los requisitos funcionales y no funcionales previamente definidos (véase Anexo B: Requisitos). Para cada objetivo, se describe lo previsto inicialmente, lo efectivamente logrado y las posibles limitaciones encontradas durante la implementación.

6.1. VERIFICACIÓN DE OBJETIVOS

Objetivo 1: Selección del *dataset*

Se ha conseguido conformar un *dataset* de calidad, equilibrado entre las categorías "neutral" y "*sexy*", con un volumen total superior a las 20.000 imágenes. La descarga se realizó mediante scripts automatizados y, posteriormente, se amplió la clase minoritaria mediante imágenes libres de Pixabay y Pexels. La revisión aleatoria de imágenes ha permitido validar la coherencia entre contenido y etiquetas.

Resultado: Objetivo completamente alcanzado.

Objetivo 2: Diseño y desarrollo de la arquitectura del modelo

Se ha realizado una comparativa entre arquitecturas propias y modelos preentrenados (MobileNetV2 y EfficientNetB3), evaluando criterios como precisión, número de parámetros y eficiencia. Finalmente, se optó por un modelo personalizado con cuatro bloques convolucionales y una capa densa optimizada, que se ajusta mejor a las limitaciones computacionales del entorno web.

Resultado: Objetivo completamente alcanzado.

Objetivo 3: Entrenamiento y optimización del modelo

Se aplicaron técnicas de mejora como *data augmentation*, ajuste de hiperparámetros, uso de *ReduceLROnPlateau* y *early stopping*. Las curvas de aprendizaje muestran una evolución estable, sin signos de sobreentrenamiento. A pesar de ciertas dificultades iniciales con versiones de Python y TensorFlow, estas fueron solventadas ajustando el entorno. Resultado: Objetivo completamente alcanzado.

Objetivo 4: Evaluación del desempeño del modelo

El modelo final ha demostrado un rendimiento notable en el conjunto de prueba (94,05 % de *accuracy* y 94,03 % de *F1-score*), con un comportamiento equilibrado entre *precision* y *recall*. La validación cruzada con k=5 ha obtenido resultados similares, confirmando la capacidad de generalización. Aunque no se alcanzaron valores absolutos del 100 %, el desempeño es robusto y consistente.

Resultado: Objetivo completamente alcanzado.

Objetivo 5: Integración en una extensión de navegador

La integración se completó con éxito en Firefox bajo *Manifest V2*, tras descartar Chrome por sus restricciones en el tratamiento de imágenes. Se cargan correctamente los archivos JSON y BIN del modelo en TensorFlow.js, y se ejecuta la predicción en tiempo real. Este proceso implicó resolver incompatibilidades entre versiones de bibliotecas y adaptar el código para el nuevo entorno.

Resultado: Objetivo completamente alcanzado, pese a requerir un replanteamiento técnico importante y pendiente de soluciones para las nuevas versiones de navegadores.

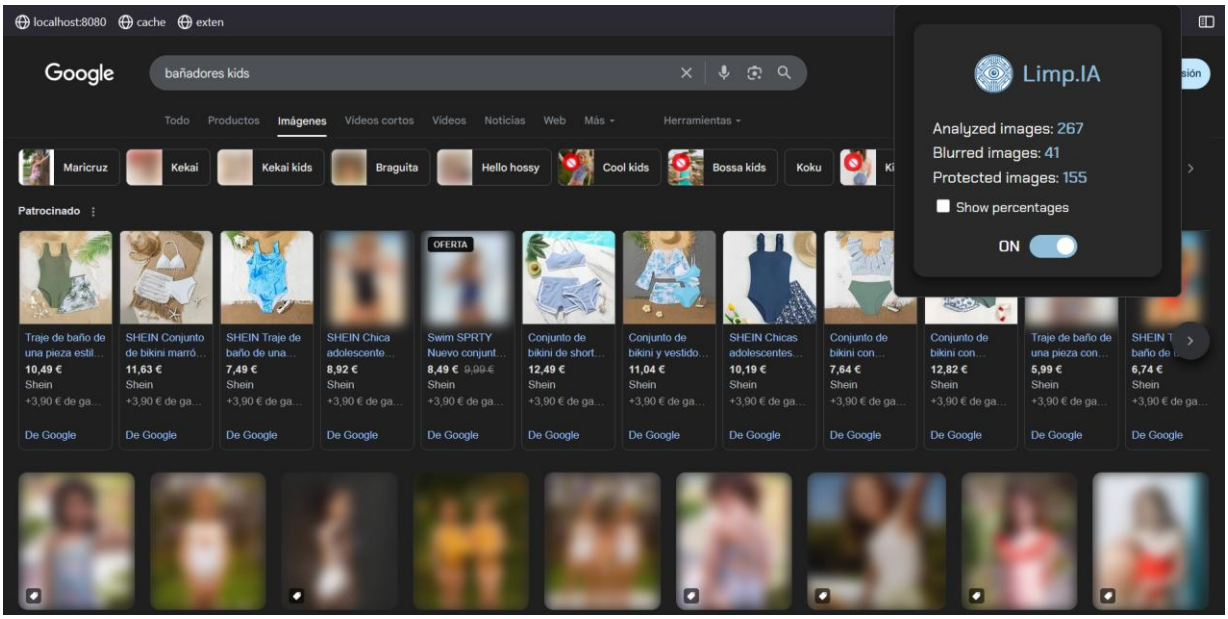

Figura 9: Limp.IA integrado en el navegador – Fuente: [29]

Objetivo 6: Pruebas en entornos reales

Las pruebas realizadas en entornos de navegación real han demostrado que la extensión funciona de manera eficaz, aplicando correctamente el difuminado en aquellas imágenes cuya probabilidad de ser clasificadas como "*sexy*" supera el 50 %. Además, el sistema permite mostrar —si así lo configura el usuario— la probabilidad estimada sobre cada imagen procesada, ofreciendo así una experiencia de usuario transparente y personalizable. El sistema ofrece una experiencia fluida y coherente con los objetivos planteados, detectando y actuando sobre el contenido sensible de forma automatizada y en tiempo real.

En determinadas páginas con un volumen elevado de imágenes, se han observado leves ralentizaciones o interrupciones puntuales en el análisis, asociadas a la carga simultánea de múltiples tensores. Asimismo, algunas imágenes no pueden ser procesadas debido a restricciones propias del navegador (como políticas CORS o limitaciones de acceso al contenido visual), pero en estos casos el sistema informa adecuadamente al usuario, garantizando la transparencia del proceso sin afectar a la funcionalidad general.

Resultado: Objetivo mayoritariamente alcanzado.

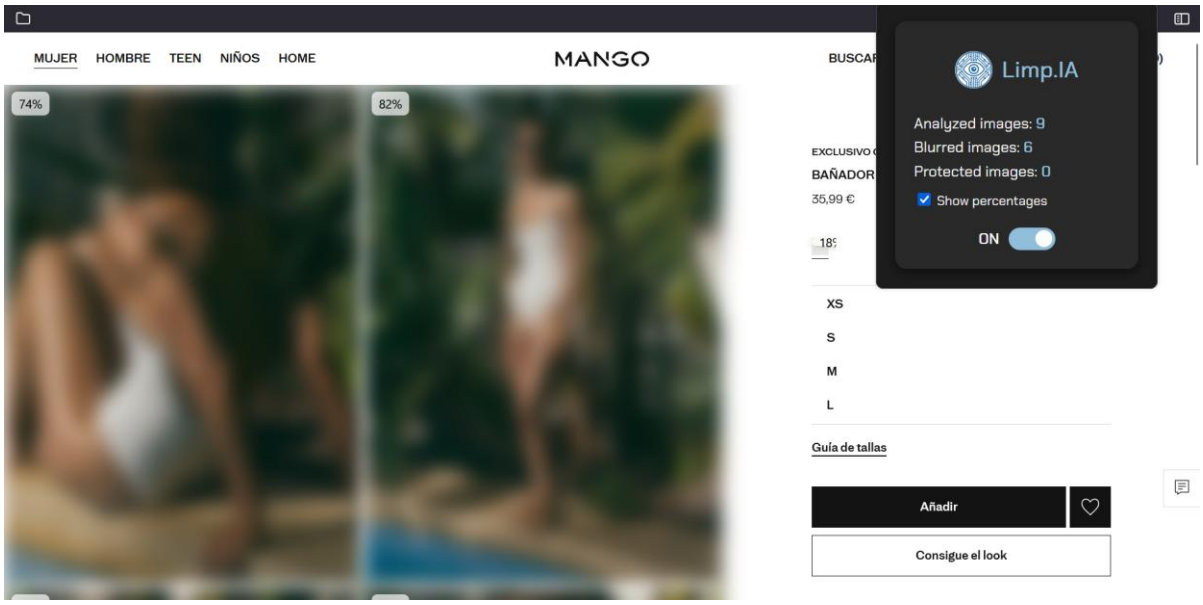

Figura 10: Limp.IA difuminando en web – Fuente: [30]

6.2. Verificación de requisitos

A partir de los resultados obtenidos, puede concluirse que el sistema desarrollado cumple la totalidad de los RF y RNF imprescindibles definidos en el Anexo B: Requisitos Además, se han implementado varias funcionalidades opcionales que, si bien no eran estrictamente necesarias para el funcionamiento básico de la extensión, aportan un valor añadido significativo al usuario al permitir una mayor personalización y ofrecer estadísticas visuales en tiempo real.

A continuación, se presenta una tabla resumen que indica el grado de cumplimiento de los requisitos definidos en el Anexo B: Requisitos, diferenciando entre aquellos implementados completamente y los que quedan pendientes para futuras versiones del sistema.

Cabe destacar que varios de los requisitos no implementados no se han descartado por inviabilidad técnica, sino por los condicionantes y limitaciones que han afectado al desarrollo del proyecto (véase Condicionantes y Limitaciones). En un escenario más favorable, estos podrían haberse abordado e incluidos en la versión actual, por lo que se consideran funcionalidades viables para futuras iteraciones.

Tabla 15: Grado de cumplimiento de los RF – Elaboración Propia

ID	Prioridad	Estado de implementación
RF-01	Imprescindible	Completado
RF-02	Imprescindible	Completado
RF-03	Imprescindible	Completado
RF-04	Opcional	Pendiente
RF-05	Opcional	Completado
RF-06	Imprescindible	Completado
RF-07	Imprescindible	Completado
RF-08	Imprescindible	Completado
RF-09	Imprescindible	Completado
RF-10	Imprescindible	Completado
RF-11	Imprescindible	Completado
RF-12	Imprescindible	Completado
RF-13	Imprescindible	Completado
RF-14	Imprescindible	Completado
RF-15	Opcional	Pendiente

Tabla 16: Grado de cumplimiento de los RNF – Elaboración Propia

ID	Prioridad	Estado de implementación
RNF-01	Imprescindible	Completado
RNF-02	Imprescindible	Completado
RNF-03	Imprescindible	Completado
RNF-04	Opcional	Completado
RNF-05	Imprescindible	Completado
RNF-06	Imprescindible	Completado

En conclusión, los resultados obtenidos reflejan un alto grado de cumplimiento respecto a los requisitos establecidos, tanto funcionales como no funcionales, especialmente aquellos marcados como imprescindibles. El sistema desarrollado no solo cumple con los objetivos técnicos propuestos, sino que demuestra su madurez y potencial para ser utilizado en escenarios reales. Representa un punto de partida robusto y escalable, sobre el que pueden construirse futuras versiones más completas.

7. IMPLICACIONES ÉTICAS E IMPACTO SOCIAL

7.1. INTRODUCCIÓN

El presente capítulo examina las implicaciones éticas y antropológicas derivadas del desarrollo de Limp.IA, una extensión para navegador basada en inteligencia artificial que detecta imágenes con contenido sugerente para proteger a los usuarios de una exposición no deseada. Enmarcado dentro de la ingeniería informática, el proyecto parte de un compromiso ético claro: proteger la dignidad de los usuarios, prevenir posibles daños psicológicos asociados a la exposición a contenido explícito y contribuir al bienestar común en el entorno digital.

7.1.1. Marco ético y legal

El marco ético principal de referencia es el Código de Ética y Conducta Profesional de la ACM [31]. A continuación, se presentan los artículos más relevantes y su aplicación a Limp.IA:

- Art. 1.1. Contribuir al bienestar humano [31].

El proyecto busca proteger a los usuarios —especialmente a menores— de la exposición involuntaria a contenido sensible, contribuyendo así a un entorno digital más seguro.

- Art. 1.2. Evitar daños [31].

Se minimizan los riesgos de daño psicológico asociados a la visualización de imágenes explícitas mediante una herramienta que actúa de forma preventiva.

- Art. 1.3 y 1.6. Ser honesto y confiable, y respetar la privacidad [31].

Limp.IA garantiza la privacidad del usuario mediante un procesamiento íntegramente local. No se almacenan ni transmiten imágenes, ni se rastrea la navegación.

- Art. 2.3. Mantener altos estándares de competencia y responsabilidad [31].

El sistema ha sido desarrollado con técnicas sólidas de IA, validado con métricas objetivas y documentado rigurosamente, asegurando calidad técnica y responsabilidad.

A ello se suma el enfoque del deontologismo moderado, que defiende el cumplimiento de principios éticos básicos —como no causar daño o respetar la privacidad—, sin olvidar las consecuencias reales de nuestras decisiones técnicas. No basta con cumplir normas: hay que reflexionar sobre cómo afectan a las personas [32].

En el plano legal, el proyecto se ajusta al Reglamento Europeo 2024/1689 sobre Inteligencia Artificial, que clasifica los sistemas según su nivel de riesgo y exige garantías de seguridad y responsabilidad [33]. Limp.IA se considera de bajo riesgo al funcionar localmente sin recopilar ni enviar datos personales.

7.2. DESARROLLO

7.2.1. Valor del proyecto

El valor central del proyecto reside en su aportación a la protección digital de los usuarios, especialmente en colectivos vulnerables como menores de edad y personas en proceso de recuperación de adicciones a la pornografía. El uso de internet conlleva el riesgo de exposición involuntaria a material sexual explícito, especialmente preocupante en adolescentes.

Diversos estudios [4] han evidenciado que dicha exposición puede afectar negativamente al desarrollo emocional y psicosexual, generando síntomas de malestar, fantasías inadecuadas o dificultades en la regulación emocional. Estos riesgos refuerzan la necesidad de sistemas proactivos de protección digital, como el propuesto en este proyecto, en línea con el deber ético de evitar daños.

A nivel profesional y social, Limp.IA constituye un ejemplo de aplicación ética de la inteligencia artificial: su diseño es transparente, su funcionamiento respetuoso con los derechos del usuario, y su código podría adaptarse a otras iniciativas de protección digital. Además, el sistema tiene un alto valor educativo y familiar, ya que permite crear espacios de navegación más seguros sin recurrir a medidas invasivas.

7.2.2. Alcance del proyecto

El proyecto se enmarca en el contexto de la navegación web cotidiana y está pensado para ser una herramienta sencilla y accesible para distintos perfiles de usuario:

- Usuarios individuales que desean evitar contenidos sexuales explícitos y mantener una experiencia de navegación respetuosa y saludable.
- Familias interesadas en supervisar de forma ética la navegación de los menores, sin recurrir a medidas intrusivas ni vulnerar la confianza entre padres e hijos.
- Profesionales de ámbitos como la educación, la psicología o el trabajo social, que pueden recomendar el uso de herramientas como Limp.IA en programas de prevención y acompañamiento en casos de adicción al contenido sexual.

El impacto directo afecta a quienes instalan la extensión, mientras que el impacto indirecto incluye la concienciación social sobre el uso saludable de internet, la reducción de la exposición involuntaria al contenido sugerente y la prevención de trastornos adictivos relacionados con el consumo de pornografía.

7.2.3. Responsabilidad e impacto

En relación con los aspectos técnicos del proyecto, se identificaron riesgos tales como:

- Fallos en la clasificación automática, que podrían dar lugar tanto a falsos positivos como a falsos negativos. Estos riesgos se han mitigado mediante un proceso continuo de mejora del modelo de clasificación. Como resultado, el sistema ha alcanzado una precisión del 94 % en el conjunto de *test*, lo que respalda su fiabilidad en casos reales.
- Sobrecarga de recursos del navegador, que podría afectar negativamente a la experiencia de navegación. Se ha optado por una arquitectura ligera y eficiente desarrollada en TensorFlow.js, con carga local de los archivos del modelo directamente en el navegador. Esto elimina la necesidad de conexión externa, reduce la latencia y garantiza un bajo consumo de recursos.
- Inaccesibilidad de algunas imágenes para su análisis. En algunos casos, el sistema no puede analizar imágenes por restricciones técnicas, como políticas CORS o formatos incompatibles. Para reducir este riesgo y mantener la transparencia, se informa al usuario mediante mensajes por consola e iconos visuales.

Más allá de lo técnico, las implicaciones sociales del proyecto son positivas: reduce la exposición a imágenes de contenido sexual, promueve una cultura de navegación responsable y puede ser especialmente útil en el ámbito escolar o terapéutico.

Desde un punto de vista económico, el proyecto no genera costes relevantes para el usuario final, ya que está basado en software libre y no requiere licencias ni servicios de pago. Desde una perspectiva medioambiental, el uso local sin necesidad de servidores externos contribuye a una menor huella energética que otras soluciones que procesan imágenes en la nube.

7.3. Conclusiones

En relación con todo lo expuesto, el análisis ético y antropológico de Limp.IA permite afirmar que el proyecto es plenamente viable y recomendable desde un punto de vista ético, legal y social. Su diseño se ajusta a los principios del Código de Ética de la ACM [31] y a los requisitos del Reglamento Europeo sobre Inteligencia Artificial [33], garantizando un equilibrio adecuado entre la protección de los usuarios y el respeto a sus derechos fundamentales.

A lo largo del desarrollo, los riesgos técnicos identificados han sido mitigados o son controlados, y el sistema se ha construido con un enfoque deontológico moderado que prioriza la dignidad humana [32]. El procesamiento local de las imágenes, sin enviar datos a servidores externos, refleja cómo las decisiones técnicas pueden guiarse por criterios éticos.

Por todo ello, el proyecto demuestra que la ingeniería informática puede y debe estar al servicio del bien común. Iniciativas como Limp.IA refuerzan la necesidad de seguir avanzando hacia un uso ético de la tecnología, que no solo responda a criterios de eficiencia o innovación, sino que ponga siempre en el centro la dignidad y el bienestar de la persona.

8. MI RECORRIDO EN LA UFV

8.1. EL PFG COMO CULMINACIÓN DE MI CAMINO UNIVERSITARIO

Cuando llegué a la universidad, lo hice lleno de motivación y con muchas ganas de comenzar la carrera que desde los 15 años tenía clara que quería estudiar. Desde entonces supe que la informática era lo que más me divertía, lo que más despertaba mi curiosidad y lo que mejor se me daba en el colegio. Mi objetivo era claro: formarme bien para ser un buen profesional y poder poner ese conocimiento al servicio de mi futura familia y de la sociedad.

Durante los primeros cursos, traté de socializar y abrirme a nuevas amistades, pero no llegué a sentir una conexión profunda con mis compañeros de clase. A eso se sumaba que seguía muy unido a mis amigos del colegio, con quienes seguía haciendo planes y compartiendo mi tiempo libre. De algún modo, me encerré en esa comodidad y rutina: asistir a clase, cumplir con lo académico, y después marcharme a casa. Salvo alguna excepción puntual —alguna charla o actividad para conseguir créditos—, mi vida universitaria en esos dos primeros años fue más bien individual.

En segundo de carrera, decidí presentarme como delegado. Esta decisión, aunque en principio parecía algo pequeño, marcó un punto de inflexión. Comencé a conocer más a fondo a mis compañeros y a relacionarme con otros cursos y grados. Esa apertura fue transformando mi visión: dejé de ver a los demás como algo ajeno o distante, y empecé a verlos como una oportunidad. Una oportunidad de crecer, de aprender, y de darme más y mejor a los demás.

Ya en tercero, esa nueva mirada me permitió vivir la universidad de una forma más plena. Empecé a participar activamente en el campus, involucrándome en actividades como la Javierada o el Hack for Good de Telefónica. Estas experiencias me aportaron conocimientos, seguridad y la oportunidad de salir de mi zona de confort, descubriendo que también fuera del aula había muchas formas de crecer como persona.

Este último año ha sido especialmente exigente. Me matriculé de 91 créditos y, durante el primer cuatrimestre, la combinación de trabajo y estudio me impidió dedicar el tiempo que habría querido a algunas asignaturas y prácticas. Aunque ha sido una etapa de mucho crecimiento personal, también ha supuesto un esfuerzo difícil de compaginar. Aun así, me he sentido muy realizado personalmente. En el segundo cuatrimestre, con un ritmo más

calmado, he podido aplicar lo aprendido y mejorar mi rendimiento, tanto individual como en grupo, disfrutando además de mis compañeros, profesores y de la vida universitaria.

Elegí este Proyecto Final de Grado porque siento una llamada profunda a ayudar a los demás. Siempre me ha movido el deseo de ser útil, aportar valor y contribuir al bien común. Desde pequeño he estado bien formado en temas relacionados con las adicciones y el uso de internet, y siempre me ha preocupado el consumo de pornografía a edades cada vez más tempranas. Creo firmemente que esto perjudica la madurez, la percepción de la realidad y el desarrollo de virtudes, haciendo a las personas más dependientes de sus instintos y menos capaces de mirar más allá de sí mismas.

Este proyecto nace de esa preocupación, pero también del deseo de formar algún día una familia con valores sólidos y disponer de herramientas útiles para su educación. El caso real de una persona traumatizada por su trabajo como moderador de contenido explícito me hizo pensar en cómo la tecnología podría evitar ese sufrimiento. Así surgió la idea de una inteligencia artificial que automatizara esa tarea, protegiendo a trabajadores, menores y familias. Esta solución no es solo un ejercicio técnico, sino también una respuesta ética y personal a un problema que me preocupa profundamente.

Con este PFG cierro mi etapa en el grado universitario sabiendo que la informática no es solo una carrera, sino una forma concreta de ayudar y servir a los demás.

8.2. VINCULACIÓN CON MI FUTURO PROFESIONAL

Este proyecto representa, de forma muy clara, aquello que me gustaría que fuera también mi futuro profesional: un camino donde la tecnología esté al servicio de las personas. He podido unir en él dos de las áreas de la informática que más me apasionan: la inteligencia artificial y el desarrollo web. Ambos campos me resultan estimulantes, tanto por sus desafíos técnicos como por el impacto real que pueden tener en la sociedad.

La inteligencia artificial me interesa especialmente por su potencial para ofrecer soluciones a problemas reales. Me motiva la idea de diseñar sistemas que mejoren la vida de las personas, que hagan su día a día más seguro, más humano y accesible. Creo que la tecnología puede tener un papel profundamente transformador si se orienta con criterio ético y sentido de servicio, y me gustaría seguir trabajando en esa dirección.

Por otro lado, el desarrollo web me atrae por su componente visual, creativo y de interacción directa con el usuario. Disfruto especialmente al diseñar interfaces funcionales y cuidadas, porque siento que construyo algo tangible que puede facilitar la vida de los demás. Este proyecto, al integrar un sistema de IA en una extensión web real, me ha permitido experimentar esa fusión entre lo técnico y la experiencia del usuario.

Pienso que este trabajo puede ser también una buena carta de presentación profesional. Refleja mis intereses, mis capacidades y el tipo de proyectos en los que me gustaría seguir creciendo. Me ha permitido no solo aplicar lo aprendido durante el grado, sino también descubrir nuevas motivaciones y confirmar que quiero orientar mi trayectoria hacia soluciones tecnológicas con impacto social.

9. CONCLUSIONES

El desarrollo de este Proyecto Final de Grado ha permitido demostrar la viabilidad de integrar modelos de inteligencia artificial en entornos cotidianos como los navegadores web, con el fin de detectar y mitigar la exposición a contenidos sensibles. Esta solución tecnológica, basada en CNNs y desplegada mediante una extensión funcional para Firefox, representa una herramienta de gran valor social y ético, aportando una respuesta innovadora a una necesidad real de protección en el uso diario de internet.

A nivel técnico, se ha logrado diseñar e implementar un sistema de clasificación de imágenes que combina precisión, eficiencia y ligereza, optimizado para su ejecución en tiempo real dentro del navegador. El modelo desarrollado, entrenado con técnicas de *data augmentation*, validado mediante *k-fold cross-validation* y convertido a TensorFlow.js, ha sido capaz de alcanzar métricas superiores al 94% de acierto, sin comprometer el rendimiento del entorno web. La elección de una arquitectura propia, frente a modelos preentrenados más pesados, ha sido clave para garantizar una ejecución ágil, especialmente relevante en dispositivos con recursos limitados.

Desde una perspectiva funcional, la extensión resultante no solo es operativa, sino que ofrece una experiencia de usuario clara, con indicadores visuales que permiten conocer el resultado de la predicción sin interferir en la navegación habitual. Este equilibrio entre funcionalidad y discreción facilita su adopción en contextos familiares, educativos o personales, donde el control sobre el contenido visual cobra especial relevancia.

El impacto potencial de esta herramienta va más allá de lo puramente técnico. Su utilidad es especialmente significativa para familias que desean establecer un entorno digital más seguro para menores, así como para personas que buscan apoyo en el control de contenidos explícitos durante su navegación. Este tipo de soluciones, fácilmente integrables y no invasivas, pueden convertirse en aliados clave en la protección digital, reforzando el acompañamiento necesario para un uso más consciente y responsable de la tecnología.

El proyecto no ha estado exento de dificultades. La adaptación a las restricciones impuestas por *Manifest V3* en Chrome, la falta de soporte en la documentación oficial de TensorFlow.js o las limitaciones derivadas de ciertas versiones de Python y NumPy supusieron desafíos técnicos importantes. Sin embargo, estas barreras se han convertido también en oportunidades para profundizar en la resolución de problemas complejos, adaptarse a entornos cambiantes y aprender a tomar decisiones críticas en contextos de incertidumbre.

La migración a Firefox fue fruto de esa capacidad de adaptación y ha permitido culminar el proyecto con una solución funcional y robusta.

En cuanto a las posibilidades de evolución futura, el sistema es escalable y flexible. Se contempla la implementación de un sistema de autenticación por contraseña para impedir su desactivación no autorizada, lo que potenciaría su valor como herramienta de control parental.

Asimismo, el sistema está planteado de forma que resulta muy sencillo ampliarlo, ya sea incorporando una clasificación multiclase o manteniendo una clasificación binaria con una ampliación de las imágenes de la categoría de contenido inapropiado. Gracias al enfoque modular del sistema y al hecho de que el *dataset* [20] ya se encuentra organizado por clases específicas, sería posible incorporar directamente las imágenes etiquetadas como "*porn*" dentro de la categoría "*sexy*". Esto permitiría al modelo distinguir entre imágenes neutras y aquellas con alto contenido corporal o sugerente, manteniendo una estructura de clasificación binaria. Esta ampliación puede realizarse fácilmente mediante la descarga automática de URLs ya etiquetadas, sin necesidad de modificar la arquitectura ni el modo de integración del modelo, lo que facilita su evolución futura y su adaptación a nuevas necesidades sin comprometer su funcionamiento actual.

Además, el enfoque adoptado ha buscado constantemente el equilibrio entre innovación y responsabilidad. Desde la selección ética del *dataset* hasta las decisiones de diseño centradas en el usuario, cada fase del desarrollo ha estado guiada por criterios técnicos y humanos. Esta mirada integral, que une precisión algorítmica con sensibilidad social, refuerza el valor del proyecto como ejemplo de cómo la tecnología puede orientarse hacia el bien.

Este proyecto ha demostrado, en definitiva, que es posible construir herramientas tecnológicas con un propósito ético, social y transformador, sin renunciar al rigor técnico ni a la eficiencia. La solución desarrollada no solo responde a un problema concreto, sino que plantea un enfoque de trabajo que puede inspirar futuras iniciativas: usar la inteligencia artificial no como fin en sí mismo, sino como medio para proteger, acompañar y mejorar la vida de las personas.

10. OTROS MÉRITOS DEL PROYECTO

Además del cumplimiento de los objetivos planteados inicialmente, el presente proyecto ha generado una serie de logros adicionales que aportan un valor significativo a nivel técnico, ético y social.

En primer lugar, cabe destacar que todo el desarrollo se ha realizado utilizando herramientas de software libre y tecnologías abiertas, como Python, TensorFlow, VS Code o Git. Esta elección no solo ha permitido una mayor flexibilidad durante el desarrollo, sino que también facilita la reutilización del proyecto por parte de otros usuarios o investigadores, fomentando así la transparencia y la accesibilidad tecnológica.

El proyecto presenta también un enfoque claramente interdisciplinar. Para su ejecución ha sido necesario integrar conocimientos de áreas diversas como la inteligencia artificial, el desarrollo web, la experiencia de usuario, la seguridad informática y la ética digital. Esta transversalidad ha permitido abordar el problema desde una perspectiva amplia y orientada a resolver una necesidad real de la sociedad actual.

Uno de los méritos más destacables es precisamente su contribución ética y social. El sistema desarrollado actúa como una herramienta de protección digital que puede ser de gran utilidad tanto para padres que desean proteger a sus hijos de contenido inapropiado, como para personas que buscan apoyo tecnológico para evitar la exposición a imágenes explícitas. Este valor añadido lo convierte en un recurso potencialmente útil en contextos educativos, terapéuticos o familiares.

Asimismo, el proyecto no se ha limitado al diseño conceptual o al entrenamiento de un modelo, sino que ha logrado su despliegue funcional real en un navegador como Firefox. Esto demuestra su aplicabilidad práctica y valida su integración en entornos cotidianos. El modelo empleado ha sido diseñado con criterios de eficiencia y robustez, lo que permite mantener una arquitectura ligera sin sacrificar rendimiento, optimizando así su funcionamiento incluso en dispositivos con recursos limitados.

Otro aspecto relevante es que el sistema completo ha sido publicado en un repositorio público de GitHub, facilitando su disponibilidad y promoviendo su posible mejora, adaptación o ampliación por parte de terceros. Esta apertura impulsa la reutilización del código como base para desarrollos futuros que compartan los mismos fines.

Precisamente, el diseño modular y la tecnología empleada abren la puerta a múltiples posibilidades de evolución futura. Entre ellas, se contempla la ampliación a una clasificación

multiclase, así como la posibilidad de mantener una clasificación binaria incorporando nuevas imágenes etiquetadas como "*porn*" dentro de la categoría "*sexy*", lo que permitiría al modelo mejorar su capacidad de detección sin modificar su arquitectura ni su modo de integración. Asimismo, se prevé la incorporación de sistemas de alertas o informes para uso parental, la implementación de un sistema de autenticación mediante contraseña que impida su desactivación no autorizada, y la adaptación a distintos navegadores si se resuelven los requisitos de compatibilidad con nuevas versiones de *Manifest*.

Finalmente, la interfaz desarrollada ha sido pensada para usuarios no expertos, con un diseño accesible, intuitivo y claro, lo que mejora notablemente la usabilidad y garantiza que el sistema pueda ser utilizado sin necesidad de conocimientos técnicos avanzados.

En conjunto, estos méritos no esperados reflejan el potencial real del proyecto más allá de su marco académico, consolidándolo como una solución innovadora, accesible y de alto valor social.

11. BIBLIOGRAFÍA

[1] S. Josemaría, Amigos de Dios nº175, Madrid: Ediciones Rialp S.A., 1977.

[2] RTVE.es / AGENCIAS, «RTVE,» 18 01 2024. [En línea]. Available: https://www.rtve.es/noticias/20240118/justicia-espanola-reconoce-problemas-salud-mental-causados-a-moderador-facebook-por-su-trabajo/15922865.shtml. [Último acceso: 23 11 2024].

[3] Ministerio de Igualdad, «Vamos a hablar de pornografía,» [En línea]. Available: https://www.igualdad.gob.es/comunicacion/campanas/vamos-a-hablar-de-pornografia/. [Último acceso: 24 11 2024].

[4] B. G. Juliá, J. Castro Calvo, E. Ruiz Palomino, M. García Barba y R. Ballester Arnal, «CONSECUENCIAS DE LA EXPOSICIÓN INVOLUNTARIA A MATERIAL SEXUAL EN ADOLESCENTES,» *International Journal of Developmental and Educational Psychology,* vol. 2, nº 1, pp. 33-44, 2018.

[5] pornblocker.org, «PorNo! Porn Blocker (Beta),» 03 05 2024. [En línea]. Available: https://chromewebstore.google.com/detail/porno-porn-blocker-beta/fnfchnplgejcfmphhboehhlpcjnjkomp.

[6] masala.one, «Smart Internet Porn Filter from vRate,» 19 09 2023. [En línea]. Available: https://chromewebstore.google.com/detail/smart-internet-porn-filte/hikjbimjpogeajolkcggpbhjdlpifcmk. [Último acceso: 05 02 2025].

[7] Cibervoluntarios, «Qué es Google SafeSearch y cómo funciona,» 09 09 2022. [En línea]. Available: https://www.cibervoluntarios.org/es/ciberacademy/post/que-es-google-safesearch-y-como-funciona. [Último acceso: 08 02 2024].

[8] Microsoft, «¿Qué es Azure Content Moderator?,» 16 10 2024. [En línea]. Available: https://learn.microsoft.com/es-es/azure/ai-services/content-moderator/overview. [Último acceso: 05 02 2024].

[9] Sightengine, «Nudity Detection,» [En línea]. Available: https://sightengine.com/docs/advanced-nudity-detection-model-2.1. [Último acceso: 05 02 2024].

[10] I. Infinite Red, «NSFW JS,» 12 2024. [En línea]. Available: https://nsfwjs.com/. [Último acceso: 05 02 2024].

[11] S. R. y. P. Norvig, Inteligencia Artificial: Un Enfoque Moderno, Pearson, 1995.

[12] d. CSV, «¿Qué es una Red Neuronal? Parte 2 : La Red | DotCSV,» 29 05 2018 . [En línea]. Available: https://www.youtube.com/watch?v=uwbHOpp9xkc. [Último acceso: 02 2025].

[13] Tensorflow, «Glosario sobre aprendizaje automático,» [En línea]. Available: https://developers.google.com/machine-learning/glossary/?hl=es-419#convolutional_neural_network.

[14] R. Rodríguez, «05_06 Deep Learning CNN Autoencoders.pdf,» 2025. [En línea]. Available: https://ufv-es.instructure.com/courses/42754/files/6050328?module_item_id=1107069.

[15] d. CSV, «¡Redes Neuronales CONVOLUCIONALES! ¿Cómo funcionan?,» 12 11 2020. [En línea]. Available: https://www.youtube.com/watch?v=V8j1oENVz00. [Último acceso: 05 02 2024].

[16] A. García, «Visión artificial con redes convolucionales (CNN),» 21 10 2019. [En línea]. Available: https://www.ellaberintodefalken.com/2019/10/vision-artificial-redes-convolucionales-CNN.html. [Último acceso: 12 02 2025].

[17] Data Science, «Convolutional Neural Network : definición y funcionamiento,» 16 12 2024. [En línea]. Available: https://datascientest.com/es/convolutional-neural-network-es. [Último acceso: 10 02 2025].

[18] Tensorflow, «Plataforma de extremo a extremo enfocada en el aprendizaje automático,» [En línea]. Available: https://www.tensorflow.org/?hl=es-419.

[19] Keras, «A superpower for ML developers,» [En línea]. Available: https://keras.io/.

[20] alex000kim, «nsfw_data_scraper,» 2019. [En línea]. Available: https://github.com/alex000kim/nsfw_data_scraper/tree/main/raw_data. [Último acceso: 15 02 2025].

[21] Pixabay, «Pixabay API documentation,» [En línea]. Available: https://pixabay.com/api/docs/. [Último acceso: 20 02 2025].

[22] Pexels, «API de imágenes y vídeos gratuitos de Pexels,» [En línea]. Available: https://www.pexels.com/api/. [Último acceso: 01 03 2025].

[23] G. C. Tech, «How to Import a Keras model into TensorFlow.js,» 26 09 2018. [En línea]. Available: https://www.youtube.com/watch?v=f8ed_xAjkOg&t=1s. [Último acceso: 02 2025].

[24] «Implementar TensorFlow.js en una extensión de Chrome,» [En línea]. Available: https://www.tensorflow.org/js/tutorials/deployment/web_ml_in_chrome?hl=es. [Último acceso: 02 2025].

[25] TensorFlow, «Importando un modelo de Keras a TensorFlow.js,» [En línea]. Available: https://www.tensorflow.org/js/tutorials/conversion/import_keras?hl=es. [Último acceso: 02 2025].

[26] Keras, «MobileNet, MobileNetV2, and MobileNetV3,» [En línea]. Available: https://keras.io/api/applications/mobilenet/. [Último acceso: 04 03 2025].

[27] Keras, «Keras Applications,» [En línea]. Available: https://keras.io/api/applications/#usage-examples-for-image-classification-models. [Último acceso: 04 03 2025].

[28] Tensorflow, «Models / Loading,» [En línea]. Available: https://js.tensorflow.org/api/latest/#Models-Loading. [Último acceso: 20 03 2025].

[29] B. d. Google, «Bañadores kids,» [En línea]. Available: https://www.google.com/search?client=firefox-b-d&sca_esv=0c6b261fdab4471f&channel=entpr&q=ba%C3%B1adores+kids&udm=2&fbs=AIIjpHx4nJjfGojPVHhEACUHPiMQWNB731WlIk-5kXFc8xThmHWnjQzcMhUOnmcqfVJMidOh0hy5PH-T4mIzXnrdovKzyhmzMypwRP-YxgB2v85gyzoRLh4DBCUjdfyuRZfP-h9. [Último acceso: 04 06 2025].

[30] Mango, «Bañador ribetes contraste,» [En línea]. Available: https://shop.mango.com/es/es/p/mujer/bikinis-y-banadores/banadores/banador-ribetes-contraste_87065951. [Último acceso: 01 06 2025].

[31] Grupo de Trabajo 2018 del Código ACM, «Código de la ACM,» 2018. [En línea]. Available: https://www.acm.org/c%C3%B3digo-de-%C3%A9tica.

[32] UFV, «Educación ética en ingeniería del software.pdf,» [En línea]. Available: https://ufv-es.instructure.com/courses/42791/files/5139647?module_item_id=979156.

[33] El Parlamento Europeo y el Consejo de la Unión Europea, «Reglamento de la IA,» 13 06 2024. [En línea]. Available: https://eur-lex.europa.eu/legal-content/ES/TXT/PDF/?uri=OJ:L_202401689. [Último acceso: 26 06 2025].

[34] Mango, «BAÑADORES DE MUJER,» [En línea]. Available: https://shop.mango.com/es/es/c/mujer/bikinis-y-banadores/banadores_79b304a0. [Último acceso: 05 06 2025].

[35] Koku, «Bañadores Niña Eco,» [En línea]. Available: https://kokufactory.com/banadores-nina-eco/. [Último acceso: 20 06 2025].

[36] Pexels, «Imágenes Gratis de Imágenes gratis,» [En línea]. Available: https://www.pexels.com/es-es/buscar/im%C3%A1genes%20gratis/. [Último acceso: 06 2025].

[37] Google, «Floristería Fia,» [En línea]. Available: https://www.google.com/search?client=firefox-b-d&sca_esv=d0afcc1efe59444d&channel=entpr&q=flores&udm=2&fbs=AIIjpHx4nJjfGojPVHhEACUHPiMQht6_BFq6vBIoFFRK7qchKMW1IFhUOYUkQKgifbEmK8vyJyQZaDHHuUNkjekqWYqSVdE4IJjj4JqXfGl6uYyi9xGQkI7GWV9E5gBtvZ5JQ8ScuFI8uHKv6DH9. [Último acceso: 20 06 2025].

[38] FreePik, «FreePik Paisajes Reales,» [En línea]. Available: https://www.freepik.es/fotos-vectores-gratis/paisajes-reales. [Último acceso: 28 05 2025].

[39] «Pixabay,» [En línea]. Available: https://pixabay.com/es/photos/flores-flores-blancas-planta-8200510/. [Último acceso: 22 05 2025].

ANEXO A: MANUAL DE INSTALACIÓN

El presente anexo proporciona una guía detallada para la instalación de la extensión Limp.IA en el navegador Mozilla Firefox. Su objetivo es facilitar al usuario el proceso de configuración inicial, asegurando que la herramienta quede correctamente instalada y lista para su uso.

Este manual incluye los pasos necesarios para habilitar el modo de desarrollador, cargar la extensión manualmente y verificar su correcto funcionamiento en el navegador. Se ha redactado pensando tanto en usuarios con conocimientos técnicos básicos como en aquellos con mayor experiencia, garantizando así una instalación accesible y eficiente.

1. Descarga la carpeta de la extensión en tu ordenador y guárdala en una ubicación fácil de recordar.
2. Abre el navegador Mozilla Firefox.

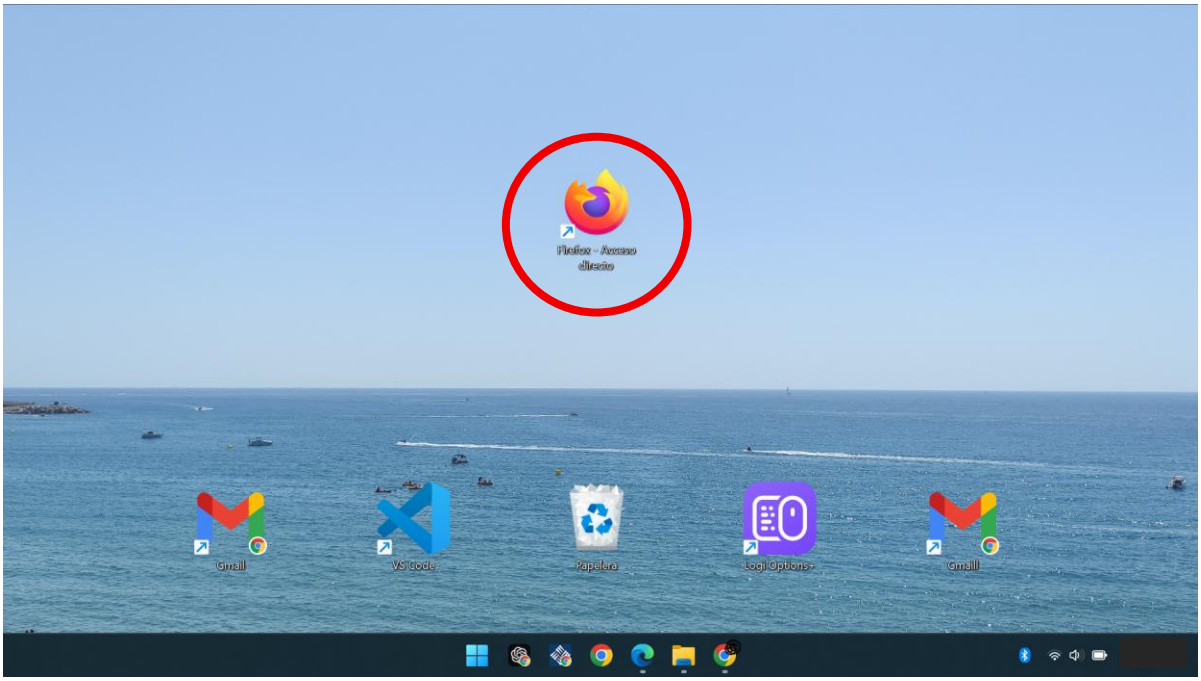

Figura 11: Abrir Firefox – Elaboración propia

3. Escribe **about:debugging** en la barra direcciones y pulsa '*Enter*'.

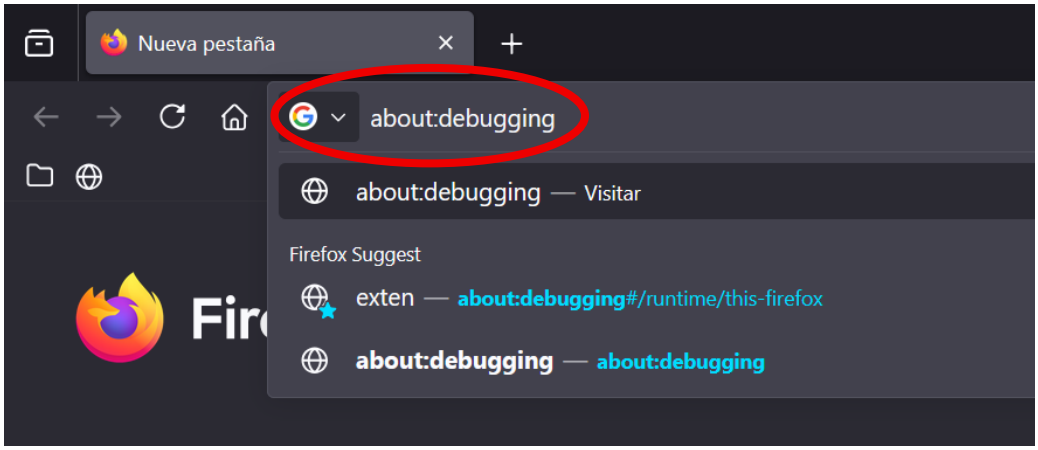

Figura 12: Buscar en la barra de direcciones – Elaboración propia

4. En el menú lateral, haz clic en **"Este Firefox"**.

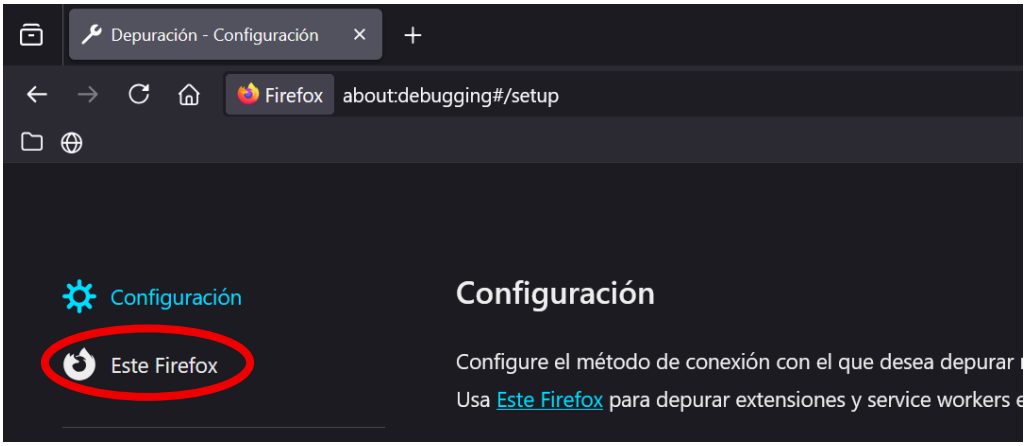

Figura 13: Menu lateral – Elaboración propia

5. Haz clic en el botón **"Cargar complemento temporal..."**.

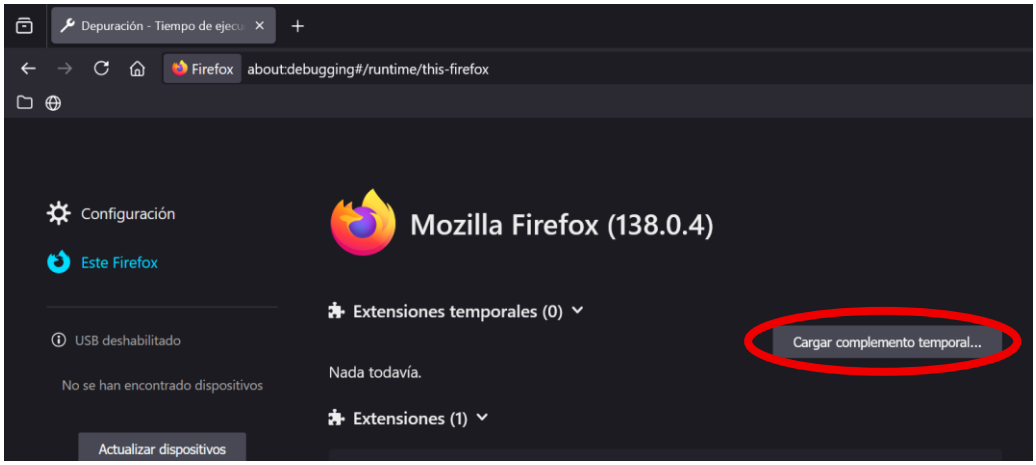

Figura 14: Cargar extensión – Elaboración propia

Nota: Recuerda que esta carga es temporal. Si cierras Firefox, deberás repetir los pasos para volver a cargar la extensión.

6. Navega hasta la carpeta donde guardaste la extensión. Una vez dentro, selecciona el archivo **manifest.json** y haz clic en 'Abrir'.

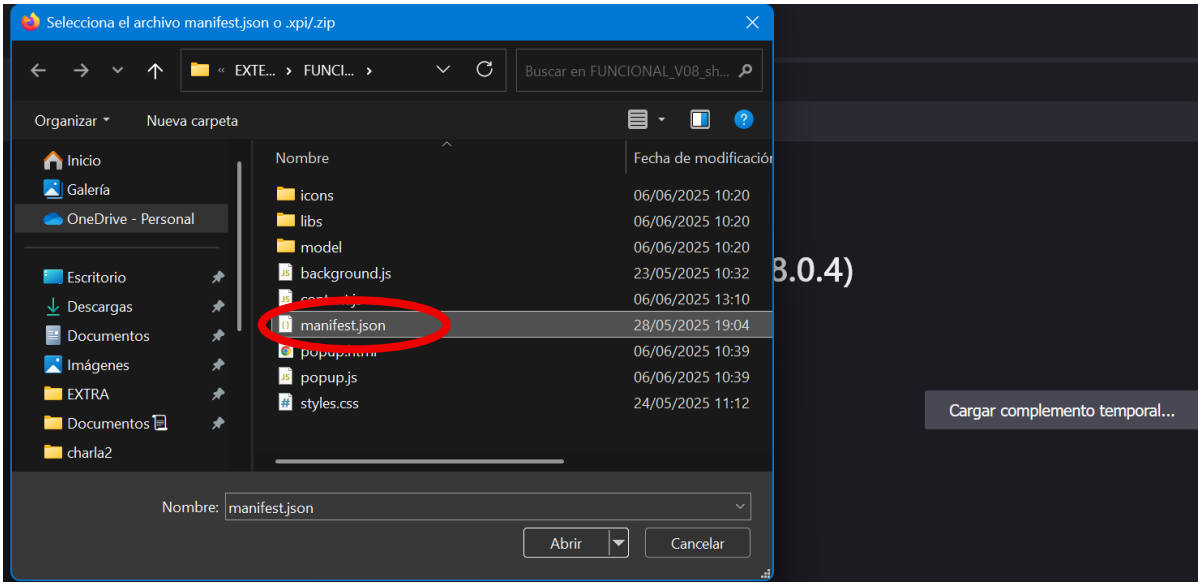

Figura 15: Seleccionar manifest.json – Elaboración propia

7. En este momento verás la extensión cargada.

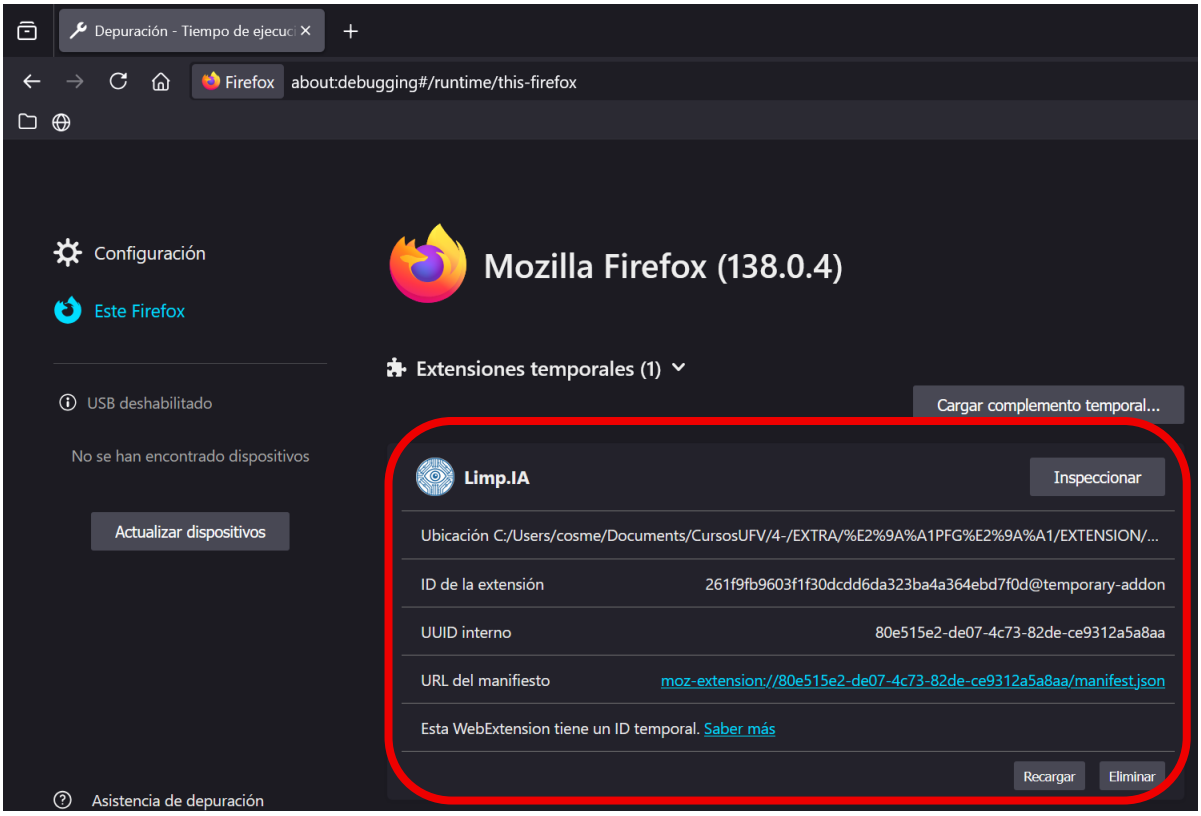

Figura 16: Extensión cargada correctamente – Elaboración propia

8. En la parte superior derecha, selecciona el icono de extensiones:

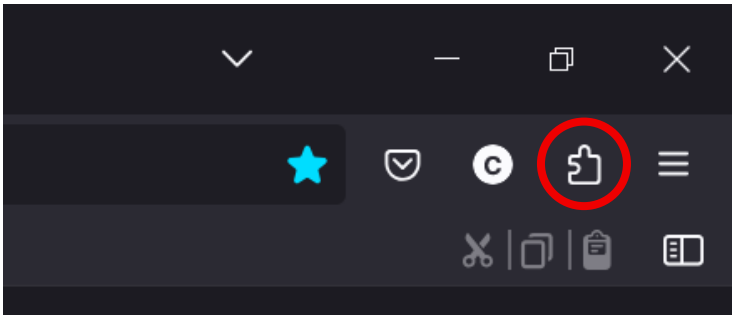

Figura 17: Icono de extensiones Firefox – Elaboración propia

9. Selecciona en los ajustes de la extensión: “Fijar a la barra de herramientas”:

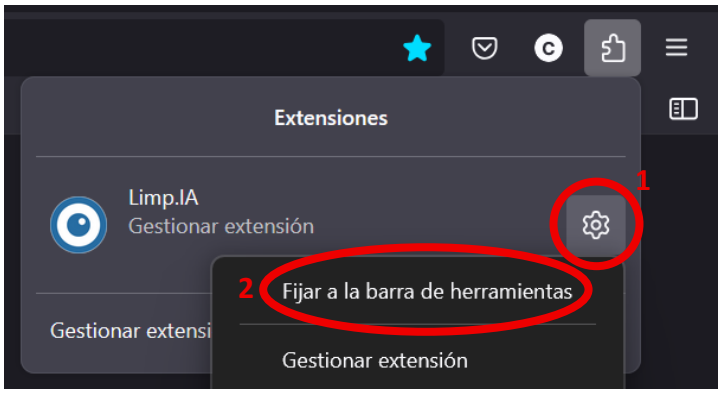

Figura 18: Fijar icono de extensión – Elaboración propia

10. ¡Listo! La extensión está activa y disponible en la barra de herramientas.

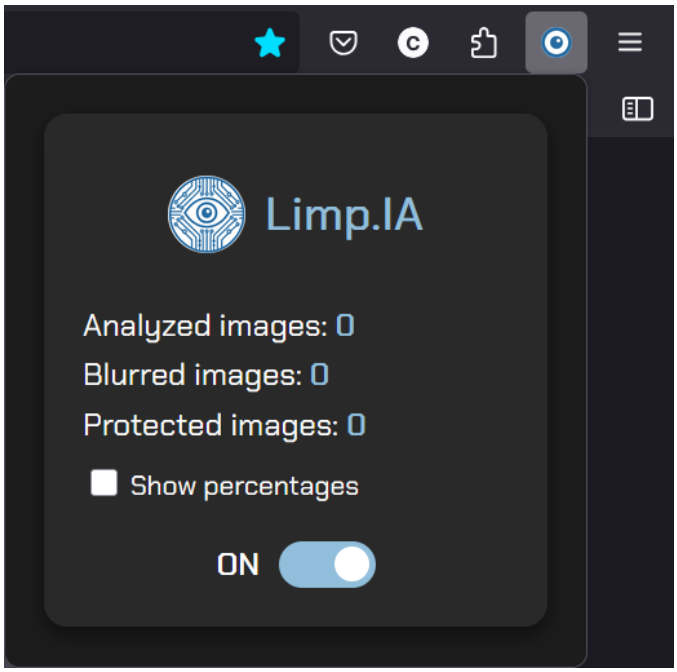

Figura 19: Popup activo y fijado en la barra de herramientas – Elaboración propia

ANEXO B: REQUISITOS

A continuación, se recogen los requisitos establecidos para el sistema desarrollado. Se han clasificado en dos categorías: RF, que definen las acciones concretas que debe realizar la aplicación, y RNF, que describen las condiciones que deben cumplirse en términos de rendimiento, calidad, seguridad o restricciones técnicas.

Cada requisito está numerado para su correcta trazabilidad y se ha asignado un nivel de prioridad, distinguiendo entre:

- Imprescindible (I): funcionalidades esenciales para el correcto funcionamiento del sistema y el cumplimiento de los objetivos del proyecto.
- Opcional (O): mejoras que, aunque no imprescindibles, aportan un valor añadido significativo en caso de poder ser implementadas dentro del tiempo y los recursos disponibles.

Tabla 17: : Requisitos Funcionales – Elaboración Propia

ID	Descripción	Prioridad
RF-01	El modelo predictivo debe ser cargado al iniciarse la extensión.	I
RF-02	El sistema debe ser capaz de detectar y clasificar imágenes asignando un porcentaje en tiempo real durante la navegación web.	I
RF-03	Cuando la probabilidad de que una imagen sea sensible supere el umbral (por defecto 50 %), la imagen debe difuminarse automáticamente.	I
RF-04	El sistema debe permitir al usuario configurar el umbral de clasificación a partir del cual se difuminan las imágenes sensibles.	O

	Este valor no podrá establecerse por debajo del 40 %, garantizando un mínimo nivel de protección.	
RF-05	El usuario debe poder decidir si quiere que se muestre el porcentaje de sensibilidad estimado sobre las imágenes analizadas.	O
RF-06	Si una imagen no puede ser procesada, el sistema debe informar al usuario con un icono visual.	I
RF-07	La extensión debe contar con un *popup* que permita activar o desactivar la extensión.	I
RF-08	El sistema debe mostrar al usuario el número de imágenes analizadas, actualizándose en tiempo real.	I
RF-09	El sistema debe mostrar al usuario el número de imágenes difuminadas, actualizándose en tiempo real.	I
RF-10	El sistema debe mostrar al usuario el número de imágenes protegidas, actualizándose en tiempo real.	I
RF-11	El sistema debe detectar y procesar imágenes cargadas dinámicamente en el DOM mediante observadores de mutaciones.	I
RF-12	El sistema debe seguir funcionando sin recargar la página, aunque se añadan nuevas imágenes mediante *scroll*.	I
RF-13	La extensión debe registrar en consola los errores internos que se produzcan sobre la carga de modelo o imágenes no accesibles.	I
RF-14	La extensión debe funcionar correctamente en cualquier página de internet con imágenes accesibles.	I
RF-15	La desactivación de la extensión solo podrá realizarse introduciendo una clave definida previamente por el usuario, actuando como mecanismo de control parental.	O

Tabla 18: Requisitos No Funcionales– Elaboración Propia

ID	Descripción	Prioridad
RNF-01	El sistema debe estar integrado en una extensión funcional para el navegador Mozilla Firefox.	I
RNF-02	El modelo debe cargarse de forma local en el navegador, sin enviar imágenes a servidores externos.	I
RNF-03	Si el modelo no puede cargarse correctamente, la extensión debe informar al usuario y desactivar su funcionalidad.	I
RNF-04	La extensión debe adaptarse a distintos tamaños de pantalla y dispositivos (PC, portátil, monitor externo).	O
RNF-05	El modelo debe de tener un porcentaje de acierto mayor al 90% en el conjunto de entrenamiento.	I
RNF-06	El sistema debe clasificar y actuar sobre cada imagen en menos de medio segundo, asegurando fluidez en la navegación.	I

ANEXO C: DESARROLLO

Este anexo recoge una selección de elementos gráficos y técnicos que ilustran el proceso de desarrollo del proyecto, complementando la documentación presentada en el cuerpo principal de la memoria. Se incluyen collages representativos de las imágenes utilizadas en el *dataset*, iconografía oficial de la extensión web y diferentes métricas visuales obtenidas durante el entrenamiento y evaluación del modelo de inteligencia artificial.

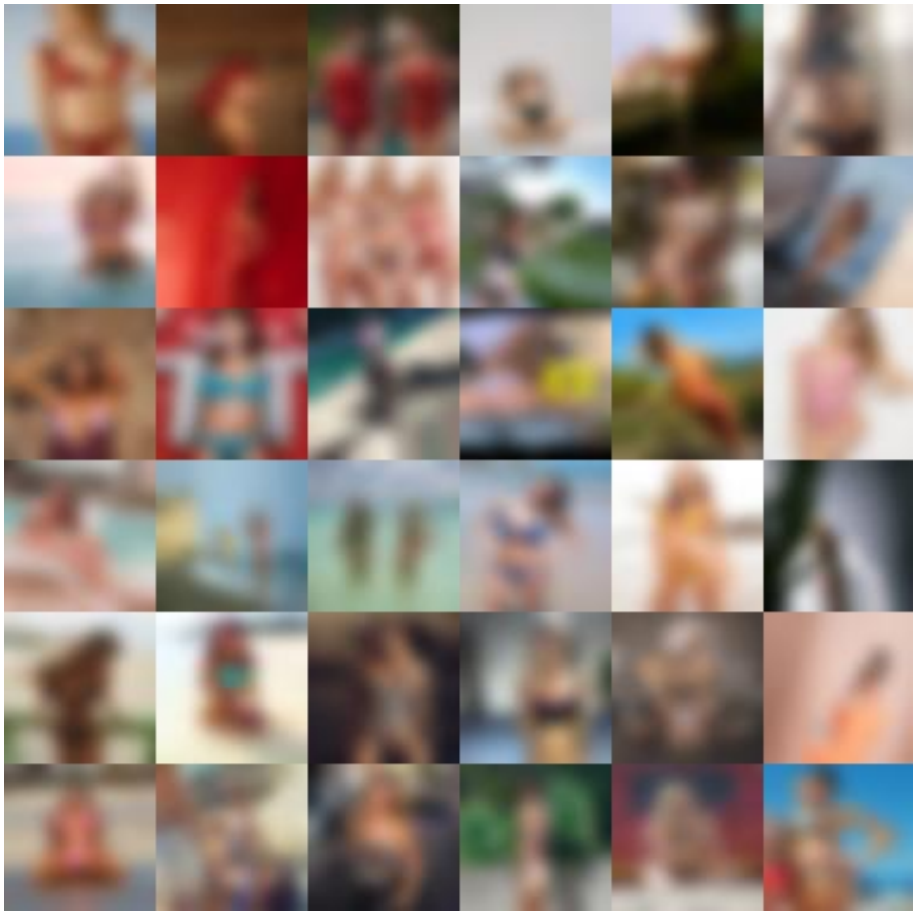

Figura 20: Muestra representativa clase sexy – Elaboración propia.

Figura 21: Muestra representativa clase neutral – Elaboración propia.

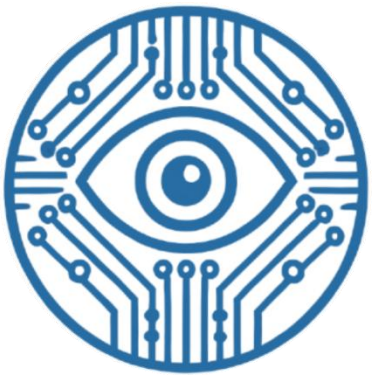

Figura 22: Icono Limp.IA – Elaboración propia.

Figura 23: Icono simplificado Limp.IA – Elaboración propia.

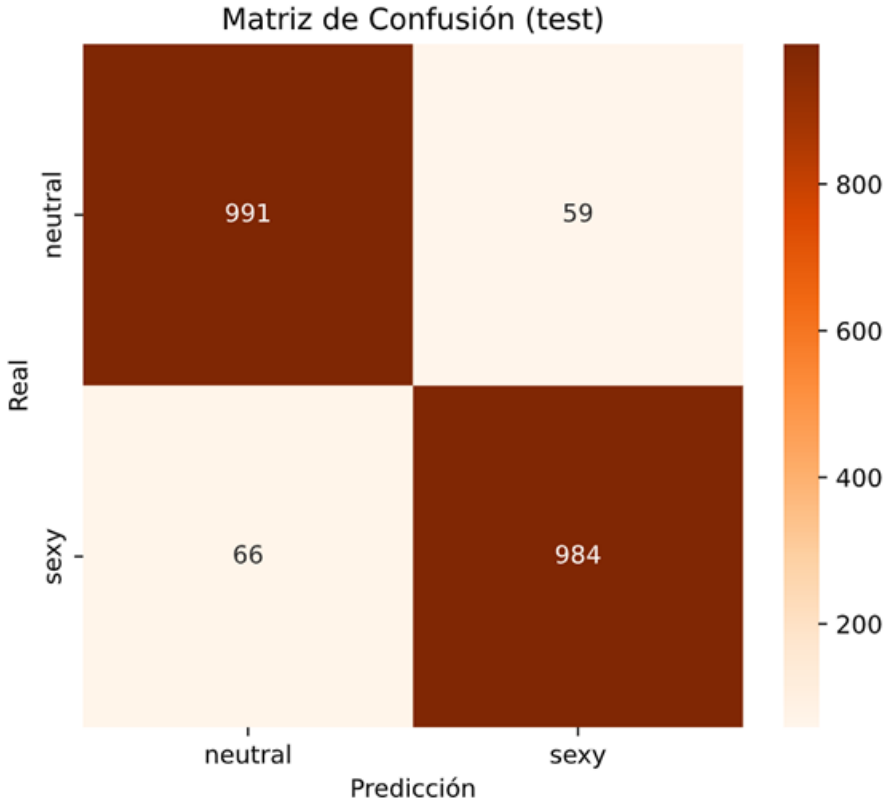

Figura 24: Matríz de confusión – Elaboración propia.

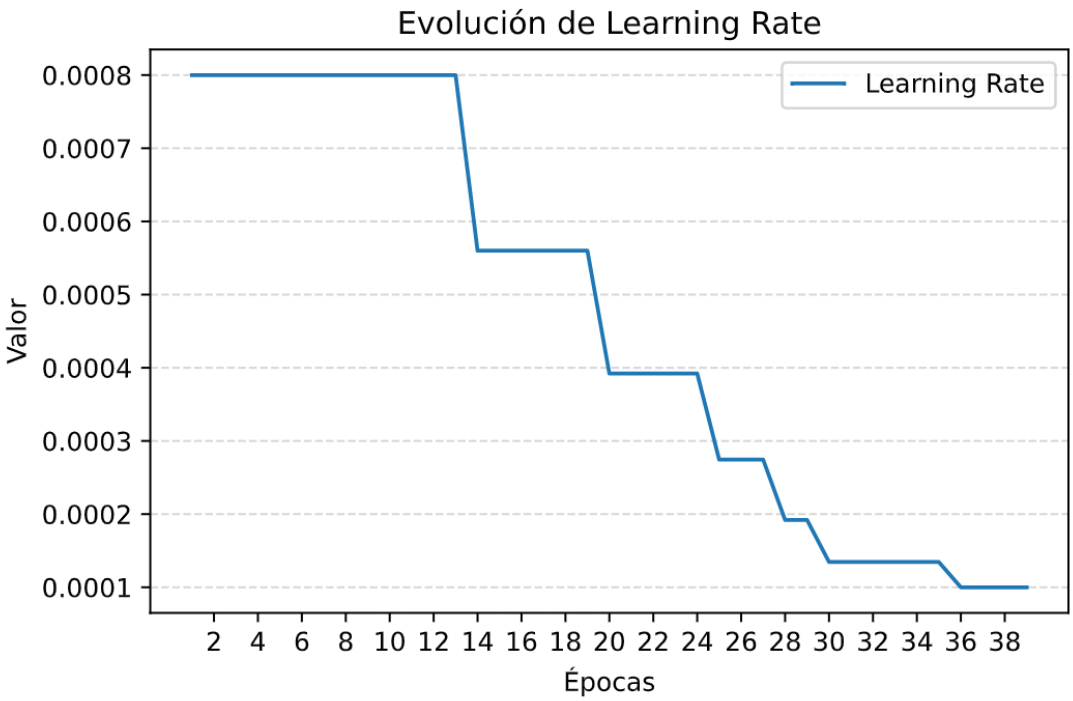

Figura 25: Evolución del LR – Elaboración propia.

ANEXO D: PRUEBAS

En este anexo se presentan ejemplos representativos de las pruebas realizadas con la extensión Limp.IA una vez completado su desarrollo e integración en el navegador. Estas pruebas permiten ilustrar de forma visual el comportamiento del sistema en distintos contextos de navegación web, verificando su capacidad para identificar, clasificar y actuar sobre imágenes con contenido potencialmente sensible.

Cada caso documentado muestra cómo el sistema procesa las imágenes, aplica el umbral de clasificación y, en su caso, difumina automáticamente aquellas que superan el umbral configurado. Además, se incluyen ejemplos en los que el sistema informa de errores o limitaciones técnicas, como la imposibilidad de analizar imágenes debido a restricciones de seguridad del navegador.

Más allá de validar el correcto funcionamiento de la herramienta, este conjunto de pruebas permite también evidenciar el potencial de Limp.IA para extenderse a otras tareas de clasificación automática en entornos web, tanto en contextos personales como profesionales. Su arquitectura modular y su funcionamiento completamente local lo convierten en una base sólida para futuras mejoras orientadas a la moderación de contenido o la protección digital.

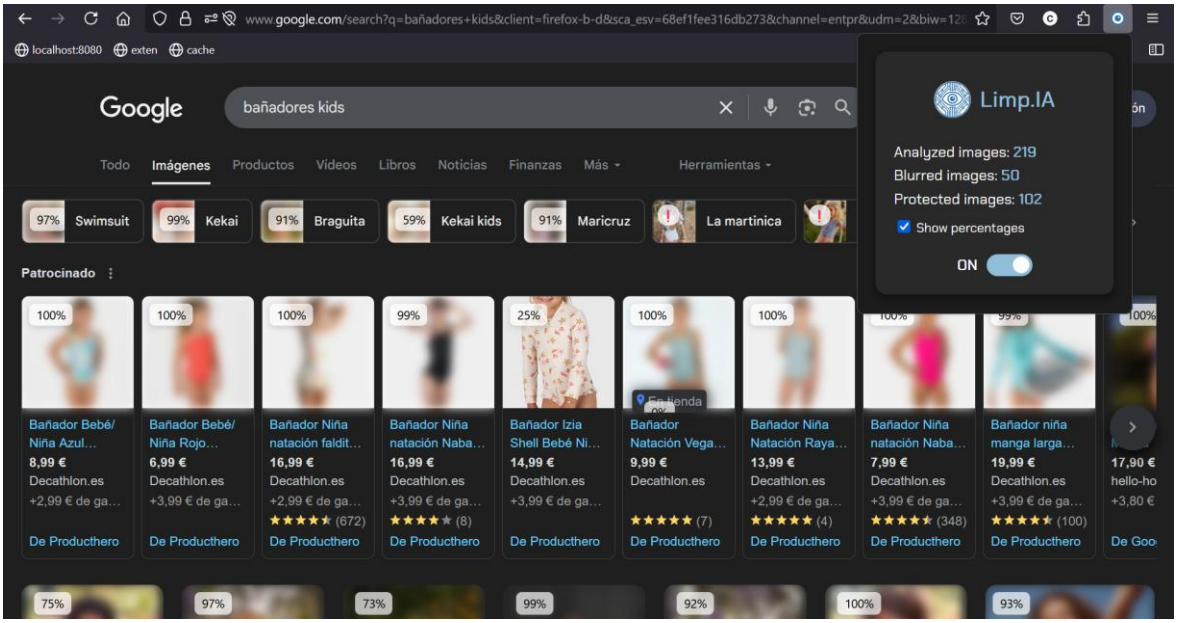

Figura 26: Limp.IA integrado en el navegador (mostrando score) – Fuente: [29]

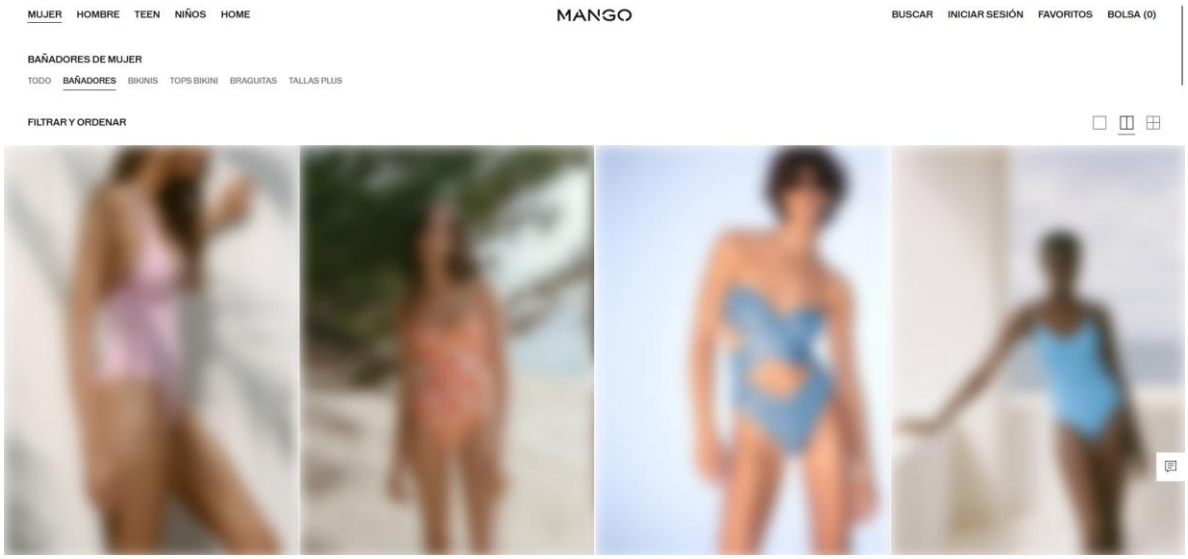

Figura 27: Limp.IA en la página web oficial de Mango – Fuente: [34]

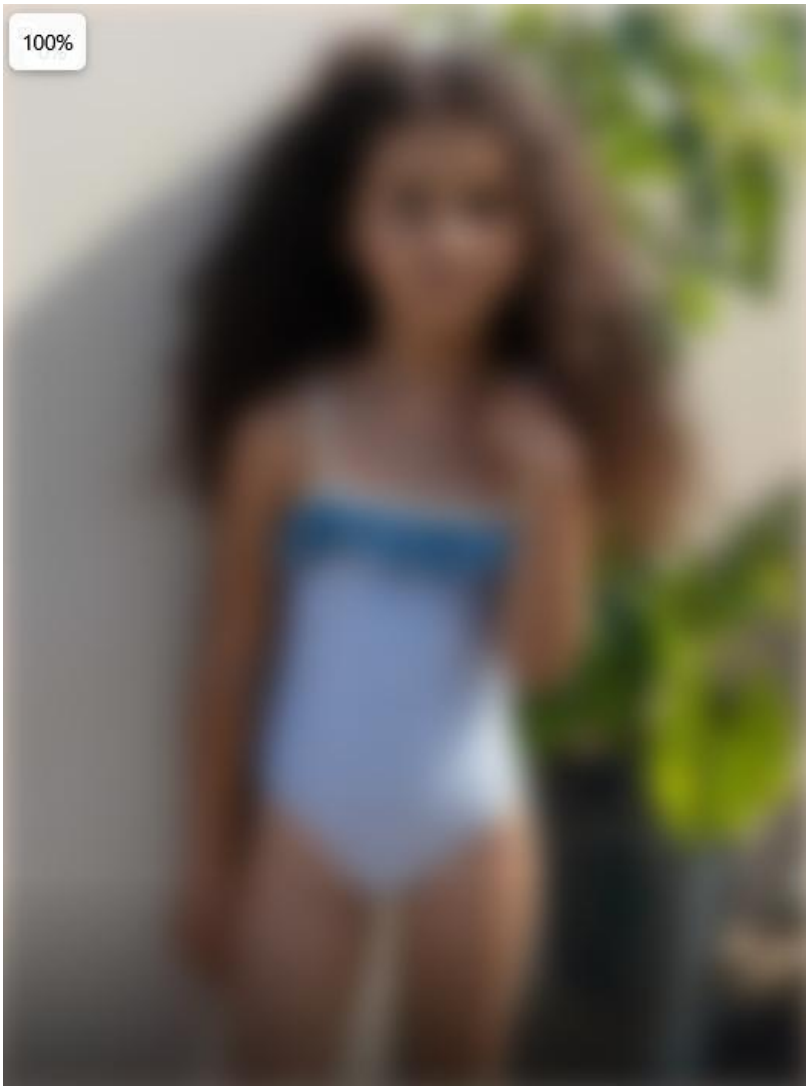

Figura 28: Ejemplo de imagen difuminada (mostrando score) – Fuente: [35]

Figura 29: Imagen no difuminada – Fuente: [36]

La imagen ha sido clasificada con un 2 % de probabilidad de contenido sensible debido a que el elemento predominante es el paisaje natural. Aunque aparece una persona en primer plano, su postura, vestimenta y el contexto general no presentan características que el modelo asocie con contenido explícito. Por ello, la puntuación asignada es muy baja.

Figura 30: Imagen no difuminada mujer – Fuente: [37]

En este caso, resulta especialmente interesante observar cómo el sistema asigna un 14 % de probabilidad de contenido sensible, únicamente por detectar la presencia de una persona en la imagen, a pesar de que las flores ocupan la mayor parte del encuadre.

Figura 31: Ejemplo imagen protegida (CORS) – Fuente: [38]

Figura 32: Ejemplo imagen no clasificada – Fuente: [39]